



실전으로 배우는 빅데이터와 AI

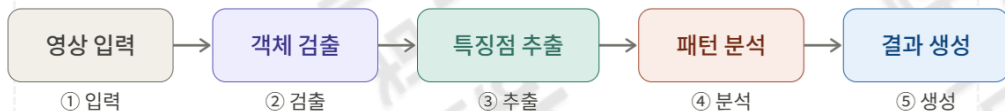
6

MediaPipe 기반 제스처 인식

• 컴퓨터 비전과 인공지능

지금까지 우리는 웹 서비스와 인공지능(GPT)을 활용하여 사용자의 텍스트 입력을 처리하고 결과를 생성하는 시스템을 구현해 보았습니다. 이번 장에서는 한 단계 더 확장하여, 카메라를 통해 입력되는 영상 데이터를 분석하고 사용자의 행동을 이해하는 컴퓨터 비전(Computer Vision) 기술을 학습합니다. 컴퓨터 비전은 이미지와 영상 데이터를 기반으로 객체를 인식하고, 특징을 추출하며, 패턴을 분석하는 인공지능 기술입니다. 이는 인간의 시각 인지 과정을 모방하여 컴퓨터가 시각 정보를 해석할 수 있도록 합니다.

컴퓨터 비전 기술은 일반적으로 다음과 같은 과정으로 구성됩니다.



• 컴퓨터 비전의 주요 기능

컴퓨터 비전 기술을 활용하면 다음과 같은 기능을 구현할 수 있습니다.

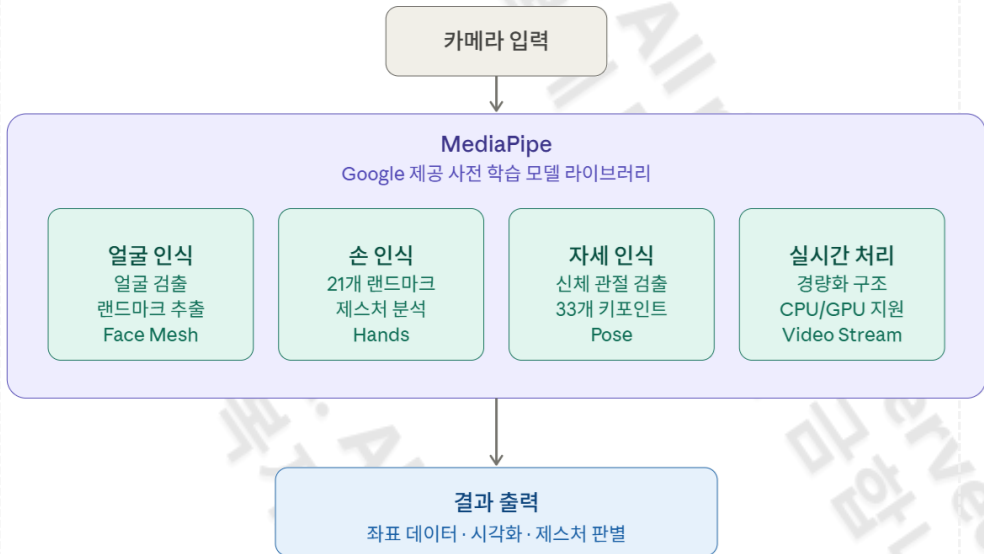
- 얼굴 검출 및 얼굴 특징점(Facial Landmark) 추출
- 손 랜드마크 기반 제스처 인식(Gesture Recognition)
- 안구 및 눈꺼풀 움직임 분석을 통한 깜빡임 검출(Eye Blink Detection)
- 인체 관절 좌표를 활용한 자세 추정(Pose Estimation)

이러한 기능은 객체 검출(Object Detection), 특징점 추출(Landmark Extraction), 그리고 패턴 인식(Recognition) 기술을 기반으로 동작합니다.

• MediaPipe를 활용한 실시간 인식

컴퓨터 비전 알고리즘을 직접 구현하는 것은 매우 복잡하지만, Google에서 제공하는 MediaPipe 라이브러리를 활용하면 고성능의 인식 기능을 비교적 간단하게 구현할 수 있습니다. MediaPipe는 다양한 사전 학습된 모델을 제공하며, 다음과 같은 기능을 실시간으로 수행할 수 있습니다.

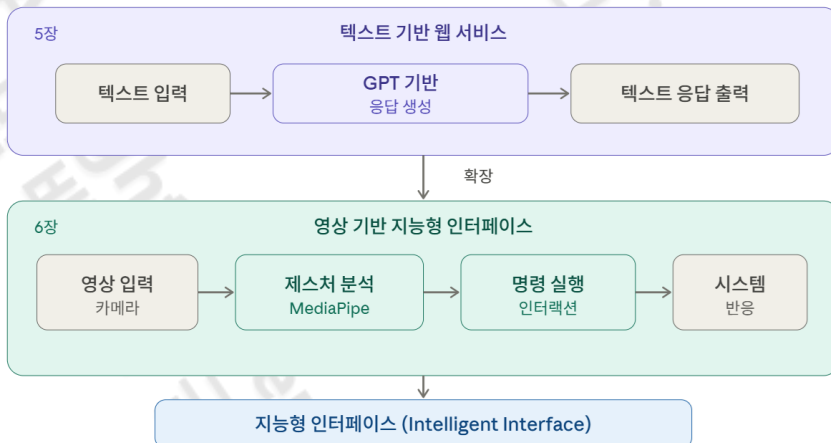
컴퓨터 비전과 제스처 인식



특히 MediaPipe는 경량화된 구조를 기반으로 설계되어 실시간 인터랙션 시스템 구현에 적합합니다.

• 웹 서비스와 컴퓨터 비전의 결합

이전 장에서 구현한 웹 서비스는 사용자의 텍스트 입력을 기반으로 동작하는 시스템이었습니다. 반면, 이번 장에서는 사용자의 행동(영상 데이터)을 입력으로 사용하여 시스템이 반응하는 구조를 구현합니다.



이러한 구조를 통해 단순한 정보 제공 시스템을 넘어, 사용자와 상호작용하는 지능형 인터페이스(Intelligent Interface)를 구현할 수 있습니다.

• 이미지 데이터 구조

컴퓨터는 이미지를 하나의 시각적 객체로 인식하는 것이 아니라, 수많은 픽셀(pixel)의 집합으로 구성된 데이터로 처리합니다. 각 픽셀은 특정 위치에 존재하는 색상 정보를 가지며, 이러한 색상은 일반적으로 RGB(Red, Green, Blue) 색상 모델을 기반으로 표현됩니다. RGB 색상 모델은 빨강, 초록, 파랑의 세 가지 색상 채널로 구성되며, 각 채널은 보통 0에서 255 사이의 정수 값으로 표현됩니다. 따라서 하나의 픽셀은 세 개의 채널 값을 갖는 벡터 형태, 즉 (R, G, B)로 나타낼 수 있습니다.

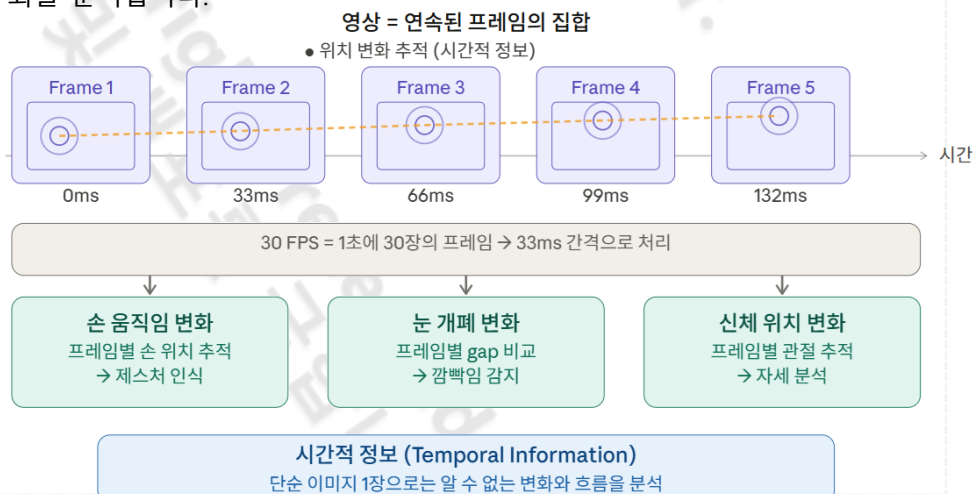
하나의 픽셀은 다음과 같은 형태로 표현됩니다.

(255, 0, 0) 빨간색	(0, 255, 0) 초록색	(0, 0, 255) 파란색	(255, 255, 0) 노란색
(0, 255, 255) 시안	(255, 0, 255) 마젠타	(255, 255, 255) 흰색	(0, 0, 0) 검정
(255, 165, 0) 주황	(128, 0, 128) 보라	(255, 192, 203) 분홍	(128, 128, 128) 회색

이미지는 이러한 픽셀들이 2차원 배열 형태로 구성된 데이터입니다.

• 영상 데이터와 프레임

영상(Video)은 연속된 이미지의 집합으로 구성되며, 각 이미지를 프레임(frame)이라고 합니다. 컴퓨터 비전에서는 이 프레임들을 순차적으로 처리하여 시간에 따른 변화를 분석합니다.



컴퓨터 비전과 영상 데이터 이해

예를 들어

- 손의 움직임 변화 → 제스처 인식
- 눈의 개폐 변화 → 깜빡임 감지
- 신체 위치 변화 → 자세 분석

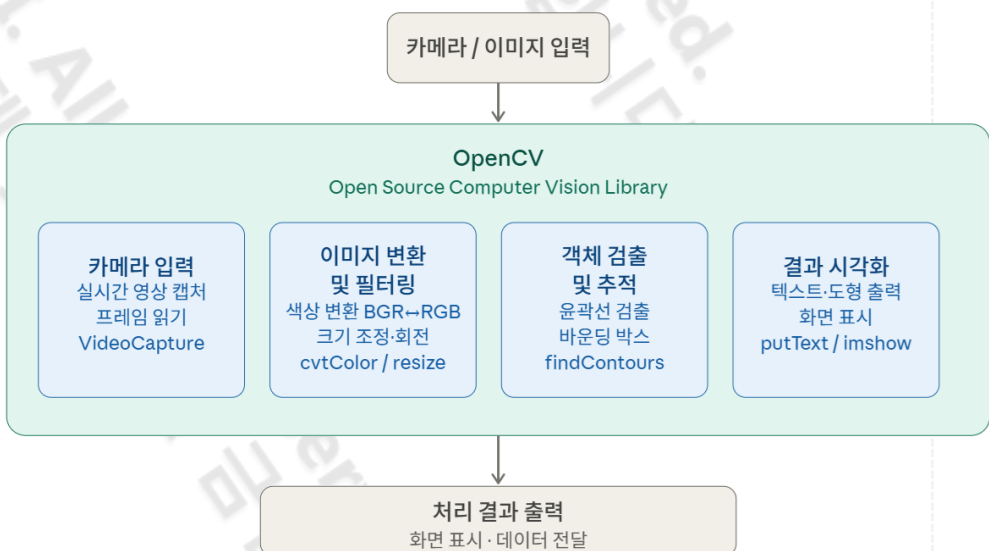
이와 같이 영상 데이터는 시간적 정보(Temporal Information)를 포함하고 있어 단순 이미지보다 더 많은 정보를 제공합니다.

• OpenCV 기초

OpenCV(Open Source Computer Vision Library)는 이미지 및 영상 처리를 위한 대표적인 라이브러리입니다.

OpenCV는 다음과 같은 기능을 제공합니다.

- 카메라 입력 처리
- 이미지 변환 및 필터링
- 객체 검출 및 추적
- 결과 시각화



랜드마크(Landmark)란?

랜드마크는 객체를 구성하는 주요 지점의 좌표를 의미합니다. 컴퓨터 비전에서는 단순히 객체의 존재 여부만 파악하는 것이 아니라, 객체 내부의 중요한 특징점을 추출하여 보다 정밀한 분석을 수행합니다.

[실시간 영상 예시 실행 코드]

```
import cv2

cap = cv2.VideoCapture(0)

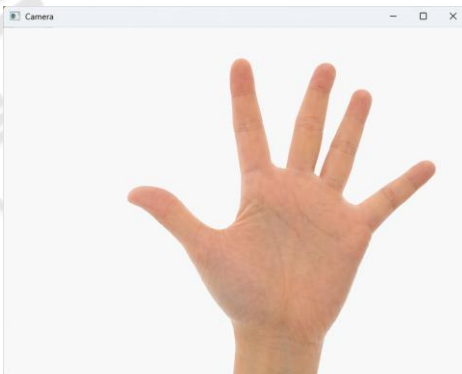
while cap.isOpened():
    success, frame = cap.read()
    if not success:
        continue

    cv2.imshow("Camera", frame)

    if cv2.waitKey(5) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

[예시 실행 결과]



[코드 설명]

- import cv2
 - OpenCV(Open Source Computer Vision) 라이브러리를 불러옴
 - 영상 처리 및 카메라 제어 기능 제공

OpenCV 기초

- `cap = cv2.VideoCapture(0)`
 - 카메라 장치를 초기화하고 영상 입력 준비
 - 0은 기본 웹캠을 의미 (환경에 따라 1, 2로 변경 가능)
 - `cap` 객체는 카메라 제어 및 프레임 읽기 역할 수행
- `while cap.isOpened( ):`
 - 카메라가 정상적으로 열려 있는 동안 반복 실행
 - 실시간 영상 처리를 위한 반복 구조
- `success, frame = cap.read( )`
 - 카메라로부터 한 프레임(frame)을 읽어옴
 - `success` : 프레임 읽기 성공 여부 (True / False)
 - `frame` : 실제 영상 데이터 (이미지 형태의 배열)
- `if not success:`
 - 프레임을 정상적으로 읽지 못한 경우 실행
- `continue`
 - 현재 반복을 건너뛰고 다음 프레임 읽기로 이동
 - 프로그램 중단 없이 안정적인 처리 유지
- `cv2.imshow("Camera", frame)`
 - 읽어온 프레임을 화면에 출력
 - "Camera"는 출력 창의 이름
 - 반복 실행을 통해 영상처럼 보이게 됨
- `cv2.waitKey(5)`
 - 5ms 동안 키보드 입력을 대기
 - 실시간 영상 출력 속도를 제어하는 역할도 수행
- `& 0xFF == 27`
 - ESC 키(아스키 코드 27) 입력 여부 확인

OpenCV 기초

- `break`
 - ESC 키 입력 시 반복문 종료
- `cap.release()`
 - 사용 중인 카메라 장치를 해제
 - 시스템 자원 반환
- `cv2.destroyAllWindows()`
 - OpenCV로 생성된 모든 창을 닫음
 - 프로그램 종료 시 필수 처리

OpenCV 라이브러리를 이용하여 웹캠으로부터 실시간 영상을 입력받고, 이를 화면에 출력하는 가장 기본적인 영상 처리 구조를 보여준다. 먼저 `import cv2`를 통해 영상 처리에 필요한 OpenCV 라이브러리를 불러온다. 이후 `cv2.VideoCapture(0)`을 사용하여 기본 카메라 장치를 초기화하고, 실시간 영상 입력을 받을 준비를 한다.

반복문 내부에서는 `cap.read()`를 통해 카메라로부터 한 장의 프레임을 읽어온다. 여기서 프레임은 영상 데이터를 구성하는 하나의 이미지이며, 이러한 프레임이 연속적으로 처리되면서 실시간 영상이 형성된다. 만약 프레임을 정상적으로 읽지 못한 경우에는 다음 반복으로 넘어가도록 하여 프로그램이 중단되지 않도록 한다.

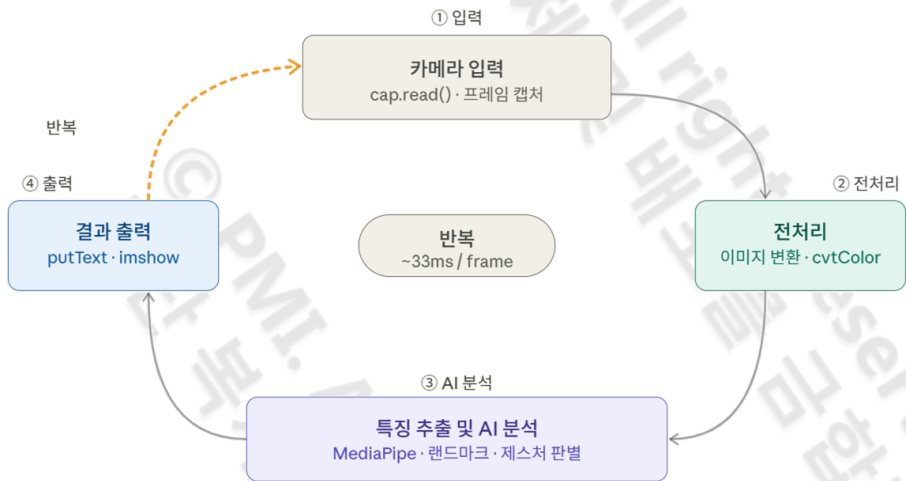
정상적으로 읽어온 프레임은 `cv2.imshow()`를 통해 화면에 출력된다. 이 과정이 매우 빠른 속도로 반복되므로 사용자는 연속적인 영상으로 인식하게 된다. 또한 `cv2.waitKey(5)`를 이용하여 키보드 입력을 감지하고, ESC 키가 눌리면 반복을 종료하도록 구성한다.

마지막으로 `cap.release()`를 통해 카메라 장치를 해제하고, `cv2.destroyAllWindows()`를 호출하여 출력 창을 모두 닫는다. 이와 같은 구조는 대부분의 실시간 컴퓨터 비전 시스템에서 공통적으로 사용되는 기본 처리 흐름이며, 이후 MediaPipe를 활용한 얼굴 인식, 손 제스처 인식, 눈 깜빡임 감지와 같은 응용 기능의 기초가 된다.

컴퓨터 비전 시스템은 일반적으로 실시간 처리(Real-time Processing)를 기반으로 동작합니다. 이는 카메라로부터 입력되는 영상 데이터를 지속적으로 처리하여, 사용자의 행동이나 환경의 변화를 즉시 분석하고 반응하는 구조를 의미합니다.

실시간 처리 구조

이와 같은 시스템은 다음과 같은 반복 처리 구조를 따릅니다.



먼저 카메라를 통해 영상 데이터가 입력되면, 해당 데이터는 분석에 적합한 형태로 변환되는 전처리 과정을 거칩니다. 이후 이미지에서 의미 있는 특징을 추출하고, 이를 기반으로 인공지능 모델이 객체 인식이나 행동 분석을 수행합니다. 분석 결과는 화면에 출력되거나 특정 동작으로 연결되며, 이러한 과정은 매우 짧은 시간 간격으로 반복됩니다. 이와 같은 반복 구조를 통해 시스템은 사용자 입력에 지연 없이 반응할 수 있으며, 손 제스처 인식, 얼굴 추적, 눈 깜빡임 감지와 같은 다양한 실시간 인터랙션 기능을 구현할 수 있습니다.

• MediaPipe의 개념

MediaPipe는 Google에서 개발한 실시간 멀티미디어 처리 프레임워크로, 이미지와 영상 데이터에서 얼굴, 손, 신체 자세와 같은 다양한 객체를 빠르게 검출하고 분석할 수 있도록 지원하는 라이브러리입니다. 특히 컴퓨터 비전과 머신러닝 모델을 결합하여 실시간 환경에서도 높은 성능을 제공하도록 설계되어 있습니다.

MediaPipe는 단순히 객체의 위치를 검출하는 데 그치지 않고, 객체를 구성하는 주요 특징점(Landmark)까지 추출할 수 있다는 점에서 매우 유용합니다. 예를 들어 손이나 얼굴을 인식하는 경우, 객체의 전체 영역뿐만 아니라 관절이나 얼굴 주요 지점과 같은 다수의 특징점 좌표를 함께 추출할 수 있습니다.

이러한 구조 덕분에 MediaPipe는 손 제스처 인식, 얼굴 특징 분석, 자세 추정, 시선 추적과 같은 다양한 응용 분야에서 활용되고 있습니다.

MediaPipe의 처리 과정은 일반적으로 Detection(검출)과 Tracking(추적)의 두 단계로 이해할 수 있습니다.

• Detection과 Tracking

1) Detection

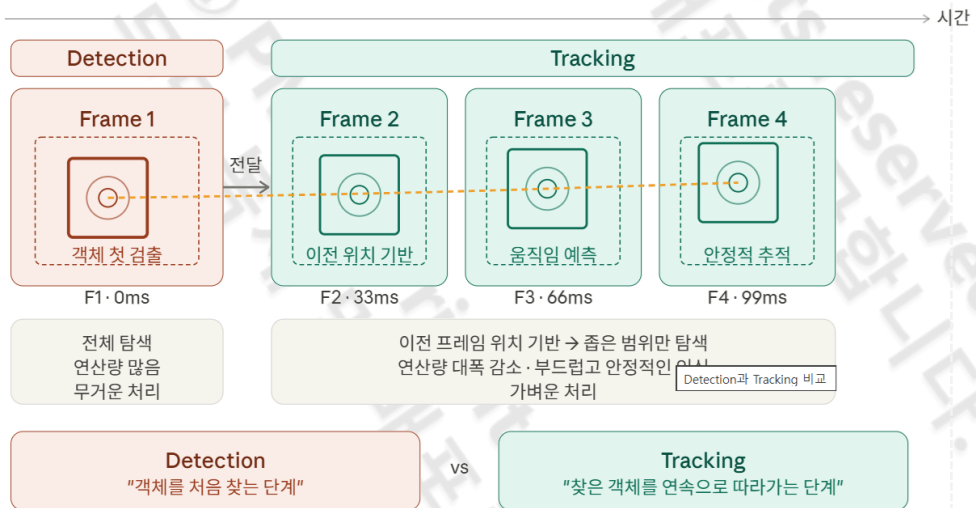
Detection은 영상이나 이미지 안에서 특정 객체가 어디에 존재하는지를 처음으로 찾아내는 과정입니다. 예를 들어, 카메라 화면 안에서 손이나 얼굴이 어느 위치에 있는지 찾는 과정이 이에 해당합니다. 이 단계에서는 객체의 전체적인 위치나 영역을 검출하는 것이 핵심입니다.

2) Tracking

Tracking은 한 번 검출된 객체를 이후 프레임들에서 계속 추적하는 과정입니다. 실시간 영상에서는 매 프레임마다 처음부터 다시 객체를 찾는 것보다, 이전 프레임에서 검출된 위치를 바탕으로 객체의 움직임을 추적하는 것이 훨씬 효율적입니다. 따라서 Tracking은 연산량을 줄이면서도 부드럽고 안정적인 인식 결과를 제공하는 데 중요한 역할을 합니다.

Detection과 Tracking

이 그림은 시간의 흐름에 따라 객체 인식 방식이 어떻게 변화하는지를 시각적으로 나타낸 것입니다. 초기 프레임에서는 객체의 위치를 찾기 위해 넓은 영역을 탐색하지만, 이후 프레임에서는 이전 위치 정보를 기반으로 탐색 범위를 줄여 객체를 추적하는 과정을 보여줍니다. 또한 프레임이 진행될수록 처리 속도가 빨라지고, 보다 안정적인 추적이 이루어지는 것을 확인할 수 있습니다

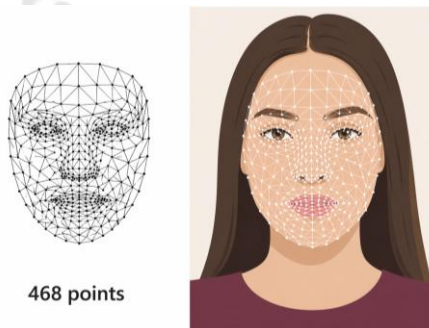


• 랜드마크(Landmark)란?

랜드마크는 객체를 구성하는 주요 지점의 좌표를 의미합니다. 컴퓨터 비전에서는 단순히 객체의 존재 여부만 파악하는 것이 아니라, 객체 내부의 중요한 특징점을 추출하여 보다 정밀한 분석을 수행합니다.

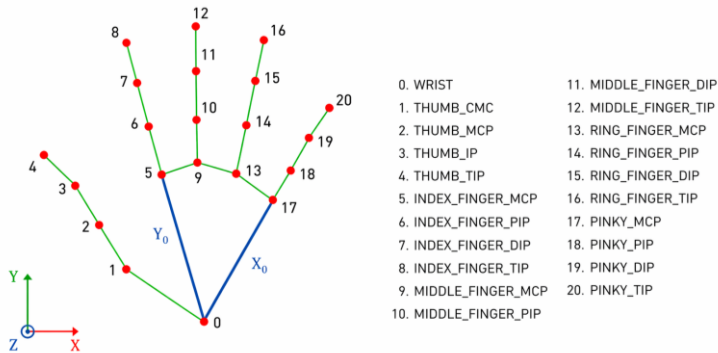
예를 들어,

- 얼굴 랜드마크는 눈, 코, 입 주변의 주요 지점을 나타내고



랜드마크(Landmark)란?

- 손 랜드마크는 손목, 손가락 마디, 손가락 끝점 등의 위치를 나타냅니다.
- 신체 랜드마크는 어깨, 팔꿈치, 무릎, 발목과 같은 관절 위치를 나타냅니다.



이러한 랜드마크를 이용하면 객체의 형태, 방향, 움직임, 자세를 더 세밀하게 분석할 수 있습니다. 예를 들어 손 랜드마크 좌표를 비교하면 손가락이 펴졌는지 접혔는지를 판단할 수 있고, 얼굴 랜드마크를 활용하면 눈의 개폐 상태나 고개 방향을 추정할 수 있습니다. 즉, 랜드마크는 객체를 단순히 "찾는 것"을 넘어, 어떤 상태에 있는지 해석하는 핵심 정보라고 할 수 있습니다.

• 좌표 (x, y, z)의 의미

MediaPipe는 각 랜드마크를 좌표 형태로 표현합니다. 일반적으로 하나의 랜드마크는 (x, y, z) 형태의 값을 가집니다.

1) x 좌표

- x 좌표는 화면에서의 가로 위치를 의미합니다.
- 값이 작을수록 화면의 왼쪽에 가깝고, 값이 클수록 오른쪽에 가깝습니다

2) y 좌표

- y 좌표는 화면에서의 세로 위치를 의미합니다.
- 값이 작을수록 화면의 위쪽에 가깝고, 값이 클수록 아래쪽에 가깝습니다.

3) z 좌표

- z 좌표는 카메라를 기준으로 한 깊이(depth) 정보를 나타냅니다.
- 즉, 객체가 카메라에 얼마나 가까운지 또는 먼지를 상대적으로 표현합니다.

다만 x, y 좌표처럼 직관적으로 보이는 2차원 좌표와 달리, z 값은 모델과 환경에 따라 상대적인 값으로 해석해야 합니다. 이와 같은 좌표 정보를 이용하면 단순한 위치 파악을 넘어, 객체의 방향성이나 입체적인 움직임까지 분석할 수 있습니다.

• MediaPipe 좌표의 특징

MediaPipe의 좌표는 일반적으로 화면 크기에 대해 정규화(normalization)된 값으로 제공됩니다. 즉, x와 y 좌표는 픽셀 단위의 절대 좌표가 아니라 0과 1 사이의 상대적인 비율 값으로 표현되는 경우가 많습니다.

예를 들어,

- $x = 0$ 에 가까우면 화면의 가장 왼쪽을 의미
- $x = 1$ 에 가까우면 화면의 가장 오른쪽을 의미
- $y = 0$ 에 가까우면 화면의 가장 위쪽을 의미
- $y = 1$ 에 가까우면 화면의 가장 아래쪽을 의미

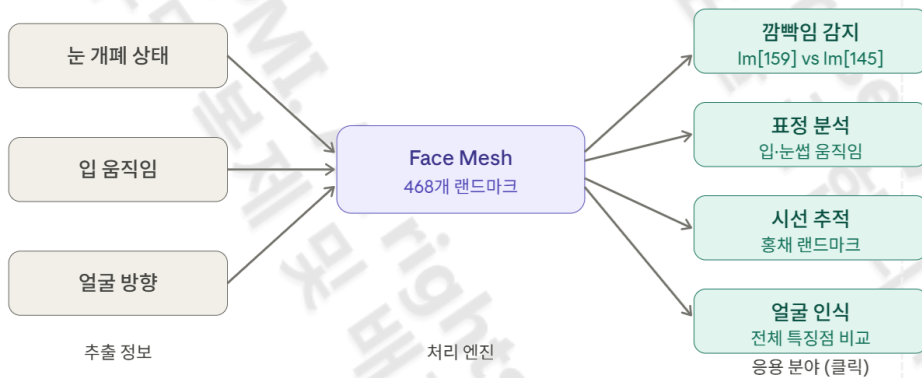
따라서 실제 픽셀 위치가 필요할 경우에는 다음과 같이 이미지 크기와 곱하여 변환할 수 있습니다.

```
x_pixel = landmark.x * image_width  
y_pixel = landmark.y * image_height
```

이러한 방식은 서로 다른 해상도의 이미지에서도 동일한 방식으로 좌표를 처리할 수 있다는 장점이 있습니다.

• Face Mesh 구조

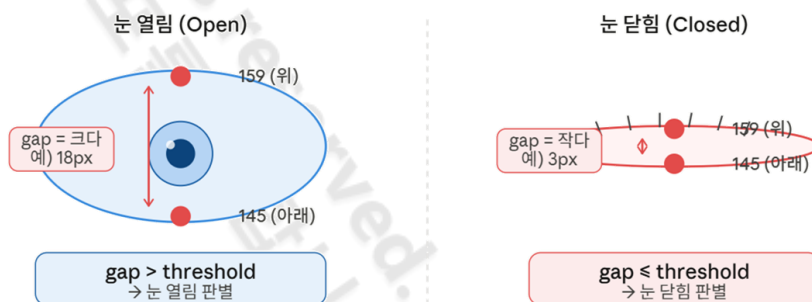
MediaPipe의 Face Mesh 모델은 얼굴의 주요 특징점을 정밀하게 추출하는 모델로, 약 468개의 랜드마크를 기반으로 얼굴의 형태를 표현합니다. 이러한 랜드마크는 눈, 코, 입, 턱선 등 얼굴의 주요 부위를 구성하는 점들로 이루어져 있으며, 이를 통해 얼굴의 구조와 움직임을 세밀하게 분석할 수 있습니다.



Face Mesh는 단순히 얼굴의 위치를 검출하는 수준을 넘어, 얼굴의 세부 특징점 좌표를 함께 제공한다는 점에서 매우 유용합니다. 따라서 눈의 개폐 상태, 입의 움직임, 얼굴 방향과 같은 다양한 정보를 분석할 수 있으며, 얼굴 인식, 표정 분석, 시선 추적, 깜빡임 감지 등의 응용에 활용됩니다.

• 눈 좌표 분석

눈 깜빡임을 인식하기 위해서는 눈 주변 랜드마크의 좌표 변화를 분석해야 합니다. Face Mesh에서는 눈의 위쪽과 아래쪽을 나타내는 랜드마크가 존재하며, 이들 사이의 거리를 계산하면 눈이 열려 있는지 닫혀 있는지를 판단할 수 있습니다.

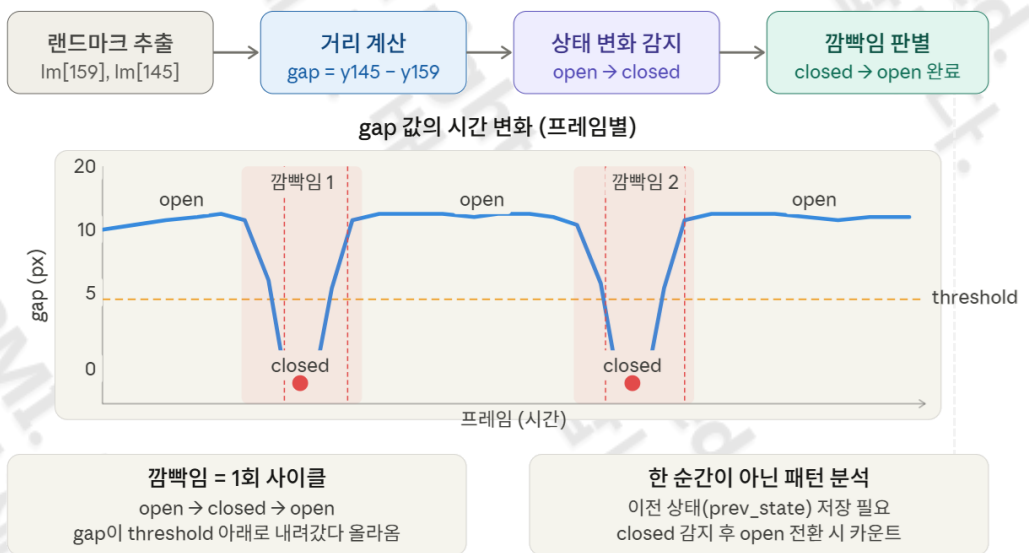


얼굴 랜드마크와 눈 깜빡임 인식

예를 들어, 왼쪽 눈의 위쪽 점과 아래쪽 점 사이의 세로 거리를 구하면, 눈이 열려 있을 때는 그 값이 커지고 눈이 감기면 그 값이 작아집니다. 따라서 눈 깜빡임 인식은 결국 눈 주변 랜드마크 좌표 간 거리 변화를 분석하는 문제로 이해할 수 있습니다. 이와 같이 계산된 거리 값(gap)을 기준값(threshold)과 비교하면 눈의 열림과 닫힘 상태를 판단할 수 있습니다.

• 깜빡임 판별 알고리즘

눈 깜빡임은 눈이 열린 상태에서 닫혔다가 다시 열리는 짧은 시간의 상태 변화를 의미합니다. 이를 정확하게 인식하기 위해서는 한 순간의 값이 아니라, 눈 상태의 변화를 시간 흐름에 따라 분석해야 합니다.



위 그림은 눈의 위·아래 랜드마크 간 거리(gap)가 시간에 따라 어떻게 변화하는지를 나타낸 것입니다. 눈이 열린 상태에서는 gap 값이 크게 유지되다가, 눈을 감는 순간 값이 급격히 감소하여 임계값(threshold) 이하로 내려갑니다. 이후 다시 눈을 뜨면 gap 값이 다시 증가하게 됩니다. 이와 같이 gap 값이 open → closed → open으로 변화하는 하나의 사이클을 하나의 깜빡임으로 판단합니다. 따라서 단순히 threshold를 기준으로 상태를 구분하는 것을 넘어, 상태 변화의 흐름을 추적하는 것이 중요합니다. 또한 이전 상태를 저장하여 closed 상태에서 open 상태로 전환되는 시점을 기준으로 깜빡임 횟수를 계산할 수 있습니다.

얼굴 랜드마크와 눈 깜빡임 인식

• 임계값 기반 판단

눈의 상태를 자동으로 분류하기 위해서는 기준이 되는 값, 즉 임계값(threshold)을 설정해야 합니다. 이 예제에서는 눈의 위아래 거리 값이 10보다 크면 OPEN, 그렇지 않으면 CLOSED로 판단하도록 구성하였습니다. 다만 눈의 크기, 얼굴의 거리, 카메라 해상도, 조명 환경에 따라 거리 값이 달라질 수 있으므로, 실제 사용 시에는 사용자의 환경에 맞게 임계값을 조정해야 합니다. 따라서 먼저 눈을 뜬 상태와 감은 상태에서 gap 값을 직접 확인한 뒤, 적절한 기준값을 정하는 것이 중요합니다.

[눈 깜빡임 인식 예시 실행 코드]

```
import cv2
from mediapipe.python.solutions import face_mesh as mp_face_mesh

cap = cv2.VideoCapture(0)
with mp_face_mesh.FaceMesh(
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5
) as face_mesh:

    while cap.isOpened():
        success, image = cap.read()
        if not success:
            continue

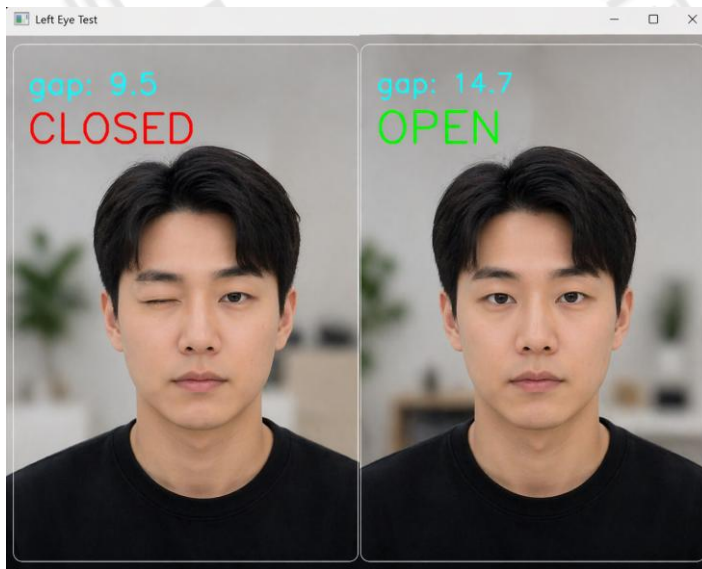
        image = cv2.flip(image, 1)
        rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        results = face_mesh.process(rgb)

        if results.multi_face_landmarks:
            for face_landmarks in results.multi_face_landmarks:
                lm = face_landmarks.landmark
                h, w, _ = image.shape
                top = lm[159].y * h
                bottom = lm[145].y * h
                gap = bottom - top
                cv2.putText(image, f"gap: {gap:.1f}", (30, 80),
                    cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 255, 0), 4)
                THRESHOLD = 10
                if gap > THRESHOLD:
                    cv2.putText(image, "OPEN", (30, 180),
                        cv2.FONT_HERSHEY_SIMPLEX, 3, (0, 255, 0), 5)
                else:
                    cv2.putText(image, "CLOSED", (30, 180),
                        cv2.FONT_HERSHEY_SIMPLEX, 3, (0, 0, 255), 5)
```


얼굴 랜드마크와 눈 깜빡임 인식

```
cv2.imshow("Left Eye Test", image)
if cv2.waitKey(5) & 0xFF == 27:
    break
cap.release()
cv2.destroyAllWindows()
```

[예시 실행 결과]



[코드 설명]

- import cv2
 - OpenCV 라이브러리를 불러옵니다.
 - 카메라 입력, 이미지 변환, 텍스트 출력, 창 표시 기능을 수행합니다.
- from mediapipe.python.solutions import face_mesh as mp_face_mesh
 - MediaPipe의 Face Mesh 기능을 불러옵니다.
 - 얼굴 랜드마크를 추출하는 데 사용됩니다.
- cap = cv2.VideoCapture(0)
 - 기본 웹캠을 열어 영상 입력을 시작합니다.
- mp_face_mesh.FaceMesh(...)
 - Face Mesh 모델을 생성합니다.

얼굴 랜드마크와 눈 깜빡임 인식

- `min_detection_confidence`는 얼굴을 처음 검출할 때의 최소 신뢰도입니다.
- `min_tracking_confidence`는 검출된 얼굴을 추적할 때의 최소 신뢰도입니다.
- `while cap.isOpened( )`:
 - 카메라가 열려 있는 동안 반복하여 프레임을 처리합니다.
- `success, image = cap.read( )`
 - 카메라로부터 한 프레임을 읽어옵니다.
 - `success`는 읽기 성공 여부, `image`는 실제 영상 데이터입니다
- `image = cv2.flip(image, 1)`
 - 화면을 좌우 반전합니다.
 - 사용자가 거울처럼 자연스럽게 볼 수 있도록 설정한 것입니다.
- `rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`
 - 화면을 좌우 반전합니다.
 - 사용자가 거울처럼 자연스럽게 볼 수 있도록 설정한 것입니다.
- `results = face_mesh.process(rgb)` : 얼굴 랜드마크 추출을 수행합니다.
- `if results.multi_face_landmarks:` : 얼굴이 검출된 경우에만 이후 코드를 실행합니다.
- `lm = face_landmarks.landmark` : 얼굴 랜드마크 정보를 간단히 사용하기 위해 변수에 저장합니다.
- `h, w, _ = image.shape` : 현재 영상 프레임의 높이와 너비를 가져옵니다.
- `top = lm[159].y * h` : 왼쪽 눈의 위쪽 랜드마크 y좌표를 픽셀 단위로 변환합니다.
- `bottom = lm[145].y * h` : 왼쪽 눈의 아래쪽 랜드마크 y좌표를 픽셀 단위로 변환합니다.
- `gap = bottom - top`
 - 눈의 위쪽 점과 아래쪽 점 사이의 세로 거리를 계산합니다.
 - 이 값이 클수록 눈이 더 많이 열린 상태를 의미합니다.

얼굴 랜드마크와 눈 깜빡임 인식

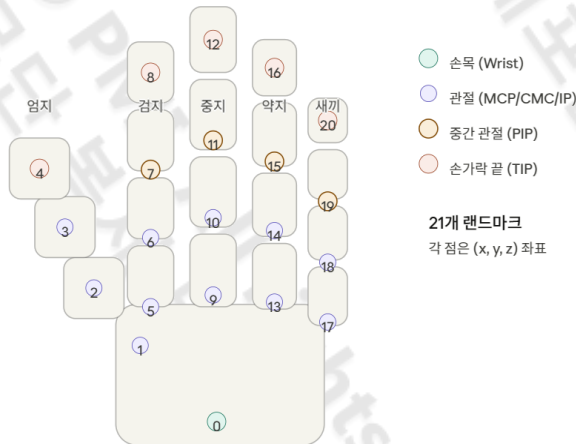
- `cv2.putText(..., f"gap: {gap:.1f}", ...)`
 - 현재 계산된 눈 거리 값을 화면에 출력합니다.
 - 사용자는 이 값을 보면서 적절한 임계값을 설정할 수 있습니다.
- `THRESHOLD = 10`
 - 눈 열림/닫힘을 구분하기 위한 기준값입니다.
 - 환경에 따라 조정이 필요합니다.
- `if gap > THRESHOLD:` : 눈 거리 값이 임계값보다 크면 눈이 열린 상태로 판단합니다.
- `else:` : 임계값 이하이면 눈이 감긴 상태로 판단합니다.
- `cv2.imshow("Left Eye Test", image)` : 결과 영상을 창에 출력합니다.
- `cv2.waitKey(5) & 0xFF == 27` : ESC 키를 누르면 반복을 종료합니다.
- `cap.release()` : 카메라 장치를 해제합니다.
- `cv2.destroyAllWindows()` : OpenCV 창을 모두 닫습니다.

이 코드를 실행하면 웹캠 영상이 실시간으로 출력되며, 얼굴이 인식되면 왼쪽 눈의 위 아래 랜드마크 간 거리(gap) 값이 화면에 표시됩니다. 사용자가 눈을 뜨고 있을 때는 거리 값이 상대적으로 크게 나타나고, 눈을 감으면 값이 작아집니다. 또한 임계값과 비교하여 눈 상태를 OPEN 또는 CLOSED로 화면에 표시하므로, 사용자는 자신의 눈 상태가 어떻게 판별되는지를 즉시 확인할 수 있습니다. 이 과정을 통해 임계값 기반의 눈 깜빡임 인식 원리를 직관적으로 이해할 수 있습니다.

손 랜드마크와 손가락 개수 인식

• Hands 구조

MediaPipe의 Hands 모델은 손의 형태를 정밀하게 분석하기 위해 총 21개의 랜드마크를 추출합니다. 이러한 랜드마크는 손목, 손가락 마디, 손가락 끝점 등 손의 주요 구조를 나타내는 특징점으로 구성되어 있습니다.



각 랜드마크는 (x, y, z) 좌표 형태로 표현되며, 이를 통해 손의 위치뿐만 아니라 손가락의 방향, 굽힘 정도, 손 전체의 형태를 분석할 수 있습니다. 예를 들어 손가락 끝점과 중간 관절의 상대적인 위치를 비교하면 손가락이 펴져 있는지 접혀 있는지를 판단할 수 있습니다.

Hands 모델은 손의 전체 영역만 검출하는 것이 아니라, 손 내부의 세부 구조를 함께 추출할 수 있다는 점에서 매우 유용합니다. 이러한 특징 덕분에 손가락 개수 인식, 제스처 분류, 손동작 기반 인터페이스 등 다양한 응용이 가능합니다.

• 손가락 펴짐 판단

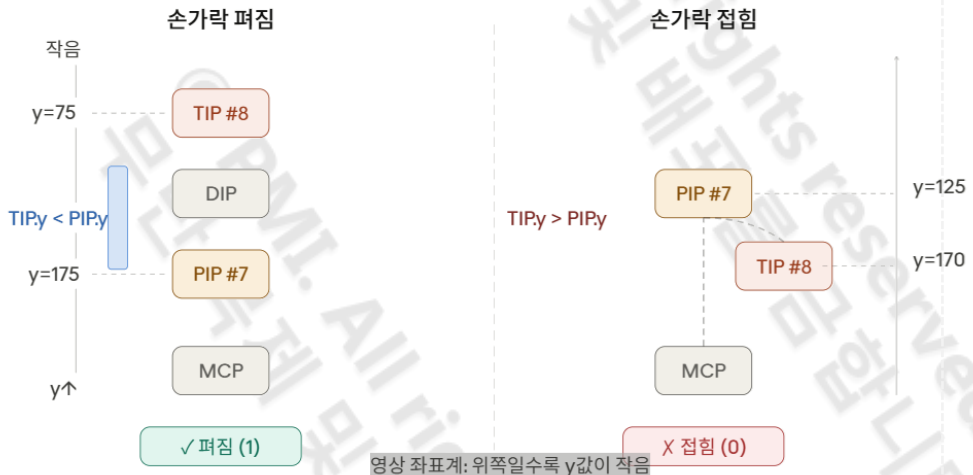
손가락 개수를 인식하기 위해서는 먼저 각 손가락이 펴져 있는지 또는 접혀 있는지를 판단해야 합니다. 이 과정은 손 랜드마크 좌표의 상대적인 위치를 비교하는 방식으로 구현할 수 있습니다.

일반적으로 검지, 중지, 약지, 새끼손가락은 손가락 끝점(TIP)의 y좌표가 중간 관절(PIP)의 y좌표보다 위쪽에 위치하면 해당 손가락이 펴져 있다고 판단합니다. 영상 좌표계에서는 위쪽일수록 y값이 더 작으므로, 끝점의 y값이 더 작으면 손가락이 펴진 상태라고 볼 수 있습니다.

손 랜드마크와 손가락 개수 인식

예를 들어 검지의 경우 다음과 같이 판단할 수 있습니다.

$$\text{INDEX_FINGER_TIP.y} < \text{INDEX_FINGER_PIP.y}$$

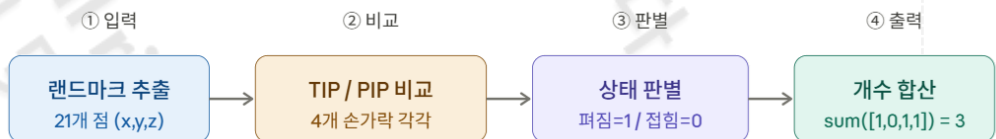


이 조건이 참이면 검지는 펴진 상태, 거짓이면 접힌 상태로 해석할 수 있습니다.

• 좌표 비교 알고리즘

손가락 상태 판단의 핵심은 랜드마크 간 좌표 비교입니다. 즉, 손가락 끝점과 중간 관절의 상대적인 위치를 비교하여 손가락이 위로 뻗어 있는지 확인하는 방식입니다.

기본적인 판단 흐름은 다음과 같습니다.



이 방식은 구현이 단순하고 실시간 처리에 적합하다는 장점이 있습니다. 다만 엄지 손가락은 다른 손가락과 방향이 다르기 때문에, 단순한 위아래 비교만으로는 정확히 판단하기 어려울 수 있습니다. 따라서 이 예제에서는 검지, 중지, 약지, 새끼손가락의 4개 손가락을 중심으로 상태를 분석합니다.

손 랜드마크와 손가락 개수 인식

• 손가락 개수 계산

각 손가락의 상태를 1(펴짐), 0(접힘)으로 표현하면, 이 값을 모두 더하여 현재 펴진 손가락의 개수를 계산할 수 있습니다.

예를 들어,

- 검지 펴짐 → 1
- 중지 펴짐 → 1
- 약지 접힘 → 0
- 새끼 접힘 → 0

이러면, 총합은 2가 되고 이는 현재 두 개의 손가락이 펴져 있음을 의미합니다.

이러한 방식은 손가락 개수 인식뿐만 아니라 이후 제스처 인식으로도 쉽게 확장할 수 있습니다. 예를 들어 손가락 개수에 따라 숫자를 입력하거나, 특정 개수 조합을 이용해 명령을 실행하는 인터랙션 시스템을 설계할 수 있습니다.

[손가락 개수 인식 예시 코드]

```
import cv2
from mediapipe.python.solutions import hands as mp_hands
from mediapipe.python.solutions import drawing_utils as mp_drawing

# 카메라 열기
cap = cv2.VideoCapture(0)

with mp_hands.Hands(min_detection_confidence=0.7, min_tracking_confidence=0.5) as hands:
    while cap.isOpened():
        success, image = cap.read()
        if not success:
            continue

        # 이미지 처리
        image = cv2.flip(image, 1) # 거울처럼 뒤집기
        rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        results = hands.process(rgb)

        if results.multi_hand_landmarks:
            for hand_landmarks in results.multi_hand_landmarks:
```

손 랜드마크와 손가락 개수 인식

```
lm = hand_landmarks.landmark

# 손가락 펴짐 감지 (TIP이 PIP보다 위에 있으면 펴진 것)
index = 1 if lm[8].y < lm[6].y else 0 # 검지
middle = 1 if lm[12].y < lm[10].y else 0 # 중지
ring = 1 if lm[16].y < lm[14].y else 0 # 약지
pinky = 1 if lm[20].y < lm[18].y else 0 # 새끼

# 손가락 갯수
count = index + middle + ring + pinky

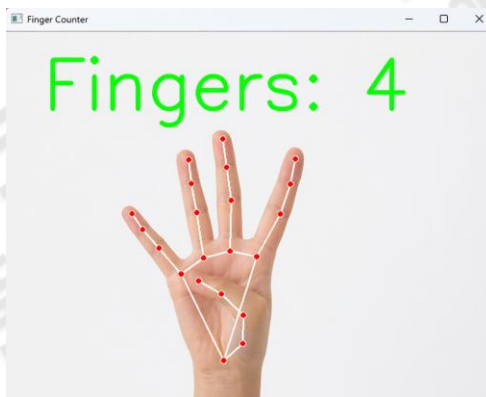
# 화면에 표시
cv2.putText(image, f"Fingers: {count}", (50, 100),
cv2.FONT_HERSHEY_SIMPLEX, 3, (0, 255, 0), 5)

# 손 뼈대 그리기
mp_drawing.draw_landmarks(image, hand_landmarks,
mp_hands.HAND_CONNECTIONS)

cv2.imshow("Finger Counter", image)
if cv2.waitKey(5) & 0xFF == 27: # ESC 키로 종료
    break

cap.release()
cv2.destroyAllWindows()
```

[예시 실행 결과]



[코드 설명]

- from mediapipe.python.solutions import hands as mp_hands
 - MediaPipe의 Hands 모델을 불러옵니다.

손 랜드마크와 손가락 개수 인식

- 손 랜드마크를 추출하는 데 사용됩니다.
- `from mediapipe.python.solutions import drawing_utils as mp_drawing`
 - 손 랜드마크와 연결선을 화면에 그리는 기능을 제공합니다.
- `cap = cv2.VideoCapture(0)`
 - 기본 웹캠을 열어 실시간 영상 입력을 시작합니다.
- `mp_hands.Hands(...)`
 - 손 검출 및 추적 모델을 생성합니다.
 - `min_detection_confidence`는 손을 처음 검출할 때의 최소 신뢰도입니다.
 - `min_tracking_confidence`는 검출된 손을 추적할 때의 최소 신뢰도입니다.
- `image = cv2.flip(image, 1)`
 - 좌우 반전된 영상을 만듭니다.
 - 거울처럼 보이도록 하여 사용자가 자연스럽게 확인할 수 있습니다.
- `rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`
 - OpenCV의 BGR 형식을 MediaPipe가 사용하는 RGB 형식으로 변환합니다.
- `results = hands.process(rgb)` : 현재 프레임에서 손 랜드마크를 추출합니다.
- `if results.multi_hand_landmarks::` 손이 인식된 경우에만 이후 코드를 실행합니다.
- `lm = hand_landmarks.landmark` : 손 랜드마크 정보를 간단히 사용하기 위해 저장합니다.
- `lm[8].y < lm[6].y`
 - 검지 끝점(TIP)이 중간 관절(PIP)보다 위에 있는지 비교합니다.
 - 참이면 검지가 펴져 있다고 판단합니다.
- `lm[12].y < lm[10].y` : 중지의 펴짐 상태를 판별합니다.
- `lm[16].y < lm[14].y` : 약지의 펴짐 상태를 판별합니다.
- `lm[20].y < lm[18].y` : 새끼손가락의 펴짐 상태를 판별합니다.
- `count = index + middle + ring + pinky` : 펴진 손가락 개수를 계산합니다.

손 랜드마크와 손가락 개수 인식

- `cv2.putText(...)` : 계산된 손가락 개수를 화면에 출력합니다.
- `mp_drawing.draw_landmarks(...)` : 손 랜드마크와 손가락 연결선을 시각화합니다.
- `cv2.waitKey(5) & 0xFF == 27` : ESC 키를 누르면 프로그램을 종료합니다.
- `cap.release()` : 카메라 장치를 해제합니다.
- `cv2.destroyAllWindows()` : OpenCV 창을 모두 닫습니다.

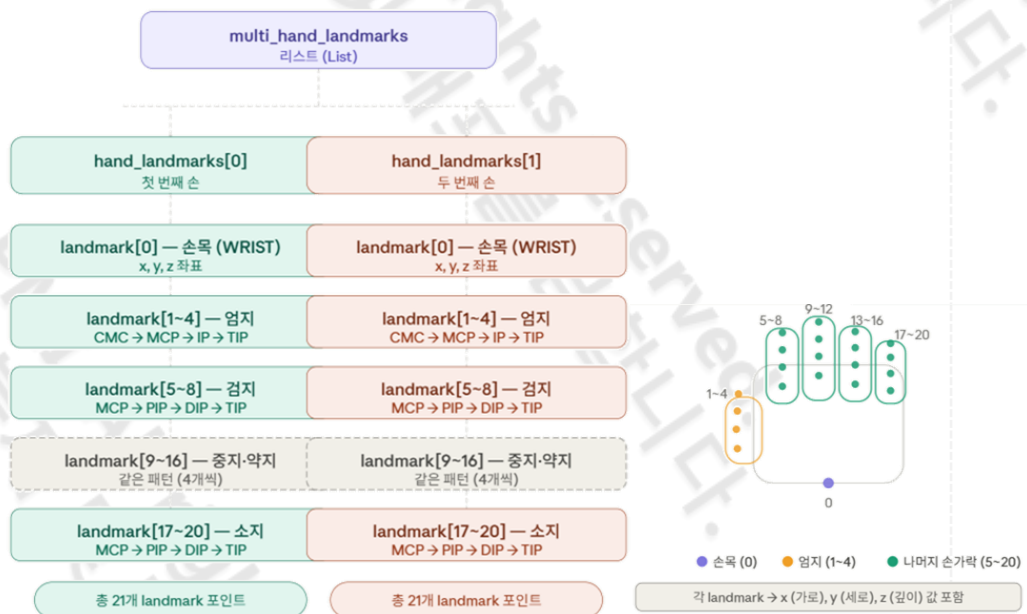
이 코드를 실행하면 웹캠 영상이 화면에 실시간으로 출력되며, 손이 인식되면 손 랜드마크가 함께 표시됩니다. 각 프레임에서 검지, 중지, 약지, 새끼손가락의 상태가 분석되며, 펴진 손가락의 개수가 Fingers: 숫자 형태로 화면에 나타납니다. 예를 들어 손가락 네 개를 펴면 화면에는 Fingers: 4가 표시되고, 두 개를 펴면 Fingers: 2가 출력됩니다. 이를 통해 사용자는 손가락 개수 인식이 어떻게 이루어지는지 직관적으로 확인할 수 있습니다.

• multi_hand_landmarks의 이해

MediaPipe Hands는 단일 손 인식에 제한되지 않고, 하나의 프레임 내에 존재하는 여러 개의 손을 동시에 검출할 수 있도록 설계되어 있습니다. 이때 각 손에 대한 랜드마크 정보는 multi_hand_landmarks라는 자료 구조에 저장되어 제공됩니다.

multi_hand_landmarks는 단일 객체가 아니라, 각 손의 랜드마크 집합을 원소로 가지는 리스트(list) 형태의 구조입니다. 즉, 하나의 프레임에서 손이 두 개 검출된 경우, 이 리스트에는 두 개의 랜드마크 데이터가 포함되며, 각각은 독립적인 손 객체를 의미합니다. 각 원소는 다시 21개의 랜드마크 좌표로 구성된 구조를 가지며, 이를 통해 개별 손의 형태와 상태를 분석할 수 있습니다.

이와 같은 구조는 다음과 같이 계층적으로 이해할 수 있습니다.



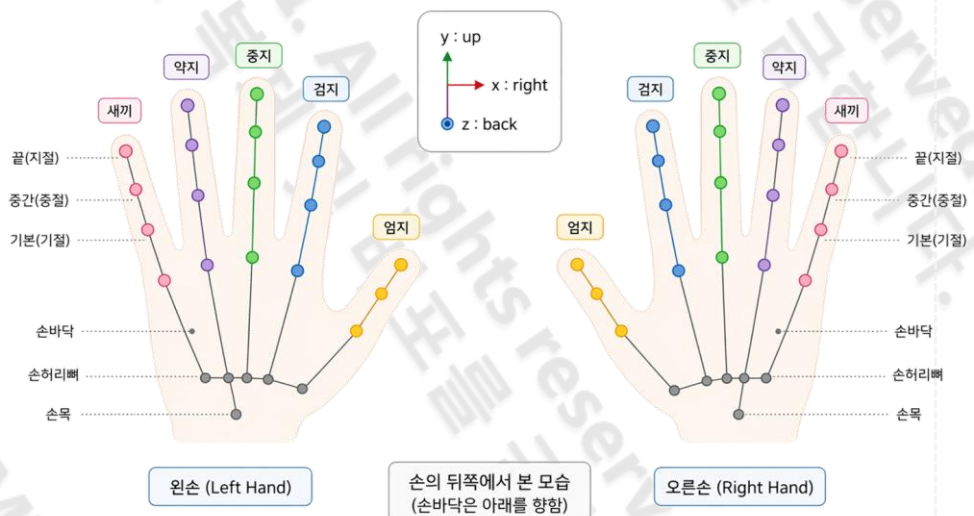
따라서 양손이 동시에 인식되는 경우, 각 손에 대해 동일한 처리 과정을 반복적으로 수행할 수 있으며, 손별로 독립적인 특징 분석이 가능합니다. 예를 들어, 한 손에서는 손가락 개수를 계산하고 다른 손에서는 특정 제스처를 판별하는 방식으로 기능을 분리하여 구현할 수 있습니다. 또한 이러한 구조는 단순한 반복 처리에 그치지 않고, 여러 손의 상태를 조합하여 보다 복잡한 입력 패턴을 설계할 수 있는 기반이 됩니다. 즉, multi_hand_landmarks는 다중 객체 인식 결과를 효율적으로 관리하고, 이를 응용 단계에서 유연하게 활용할 수 있도록 하는 핵심 데이터 구조라고 할 수 있습니다.

양손 인식 및 제스처 확장

• 왼손과 오른손 구분 및 역할 분담

양손 기반 인터랙션 시스템을 구현할 때는 단순히 두 손이 인식되었다는 사실만으로는 충분하지 않다. 각 손이 왼손(Left Hand)인지 오른손(Right Hand)인지 구분되어야, 손별로 서로 다른 기능을 부여할 수 있기 때문이다.

MediaPipe Hands는 각 손의 랜드마크 정보와 함께 손의 방향 정보(handedness)를 제공하며, 이를 통해 왼손과 오른손을 식별할 수 있다. 이러한 정보는 양손 제스처 기반 인터페이스를 설계하는 데 핵심적인 역할을 한다.



위 그림에서 확인할 수 있듯이, 각 손은 동일하게 21개의 랜드마크 구조를 가지며, 각 랜드마크는 x축: 좌우 방향, y축: 상하 방향, z축: 깊이 방향 (카메라 기준 앞/뒤)으로 표현된다. 또한 handedness 정보를 함께 활용하면, 동일한 구조를 가진 두 손이라도 왼손과 오른손을 정확하게 구분할 수 있다.

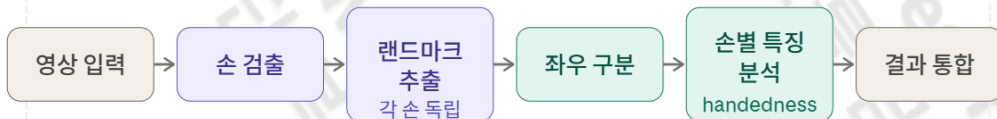
왼손과 오른손을 구분하면 각 손에 서로 다른 기능을 부여하여 인터페이스를 설계할 수 있다. 예를 들어, 왼손은 기능을 선택하거나 모드를 전환하는 역할을 담당하고, 오른손은 선택된 기능을 실행하거나 세부 명령을 입력하는 역할을 수행하도록 구성할 수 있다. 이와 같은 구조는 한 손이 입력의 의미를 정의하고 다른 손이 실제 동작을 수행하는 방식으로 동작하여, 보다 직관적이고 효율적인 사용자 인터페이스를 구현할 수 있게 한다.

양손 인식 및 제스처 확장

• 양손 동시 처리

양손이 동시에 인식되는 경우에는 하나의 프레임 안에서 검출된 모든 손에 대해 동일한 분석 과정을 반복적으로 수행해야 한다. 즉, 시스템은 단일 손을 처리하는 방식과 동일한 절차를 각 손에 독립적으로 적용하고, 그 결과를 통합하여 최종 동작을 결정한다.

이 과정은 일반적으로 다음과 같은 흐름으로 이루어진다.



각 손은 multi_hand_landmarks 구조에 저장된 개별 객체로 처리되며, 반복문을 통해 순차적으로 접근하여 분석이 이루어진다. 이때 각 손의 랜드마크 좌표를 이용하여 손가락의 상태(펴짐/접힘), 특정 제스처 여부, 위치 정보 등을 독립적으로 계산할 수 있다. 예를 들어, 한 프레임에서 두 손이 동시에 인식된 경우, 왼손에서는 펴진 손가락의 개수를 계산하고 오른손에서는 특정 제스처를 판별하는 방식으로 서로 다른 분석을 수행할 수 있다. 이후 이러한 결과를 결합하여 하나의 의미 있는 명령으로 해석한다. 예를 들면, 왼손의 손가락 개수를 숫자 입력으로 사용하고, 오른손의 제스처를 실행 명령으로 사용하는 구조를 구성할 수 있다. 이처럼 각 손의 분석 결과를 조합하면 단일 손 인식에서는 표현하기 어려운 복합 입력을 구현할 수 있다.

• 제스처 패턴 설계

양손 인식의 핵심적인 장점은 두 손의 상태를 조합하여 하나의 의미 있는 동작으로 해석할 수 있는 복합 제스처 패턴을 정의할 수 있다는 점이다. 단일 손 인식이 개별 손의 상태만을 기반으로 동작을 판단하는 것과 달리, 양손 인식은 두 손의 관계를 활용하여 보다 다양한 입력을 표현할 수 있다. 예를 들어, 왼손이 주먹 상태이고 오른손이 검지를 펴고 있는 경우 이를 "시작" 동작으로 정의할 수 있으며, 왼손에서 두 개의 손가락이 펴지고 오른손이 모두 펼쳐진 상태를 "모드 전환"으로 해석할 수 있다. 또한 양손을 모두 펼친 상태는 "초기화", 양손을 모두 주먹으로 만든 상태는 "종료"와 같이 다양한 패턴을 설계할 수 있다. 이와 같은 제스처는 각 손의 상태를 독립적으로 분석한 뒤, 두 결과를 조건식으로 결합하여 하나의 동작으로 해석하는 방식으로 구현된다. 즉, 제스처 인식의 기준이 단일 손의 상태가 아니라, 두 손의 조합 관계에 의해 결정된다.

양손 인식 및 제스처 확장

• 제스처 설계 시 고려 요소

양손 제스처를 효과적으로 설계하기 위해서는 다음과 같은 요소를 함께 고려해야 한다.

첫째, 각 손의 개별 상태이다. 각 손에서 손가락의 펴짐 여부, 특정 손 모양(주먹, 펼침 등), 위치 정보 등을 정확하게 판단할 수 있어야 한다.

둘째, 두 손 상태의 조합 규칙이다. 동일한 손 상태라도 다른 손과 어떻게 결합되는지에 따라 전혀 다른 의미를 가질 수 있으므로, 두 손의 관계를 명확하게 정의해야 한다.

이처럼 양손 제스처 인식은 단순한 상태 판별이 아니라, 두 입력 간의 관계를 해석하는 과정으로 이해할 수 있다

• 양손 인식의 활용 예시

양손 제스처 인식은 다양한 인터페이스 시스템에 적용될 수 있으며, 단일 손 인식보다 훨씬 풍부한 입력 구조를 제공한다. 예를 들어, 한 손은 숫자 입력을 담당하고 다른 손은 실행 명령을 담당하도록 구성할 수 있으며, 한 손은 방향을 제어하고 다른 손은 속도를 조절하는 방식으로도 활용할 수 있다. 또한 두 손의 조합을 이용하여 모드를 전환하거나, 특정 제스처 패턴을 통해 프로그램을 제어하는 등의 응용이 가능하다.

이러한 구조는 사용자와 시스템 간의 상호작용을 보다 직관적이고 유연하게 만들어 주며, 복잡한 입력을 간단한 동작으로 표현할 수 있도록 한다.

• 코드로 구현하기

지금까지 살펴본 양손 인식과 제스처 패턴 설계 개념을 실제 코드로 구현하면, 각 손의 랜드마크를 기반으로 손가락 상태를 분석하고, 이를 바탕으로 제스처를 판별할 수 있다.

아래 코드는 MediaPipe Hands를 활용하여 양손을 동시에 인식하고, 각 손의 손가락 상태를 분석한 뒤, 제스처와 손가락 개수를 화면에 출력하는 예제이다.

[양손 인식 예시 코드]

양손 인식 및 제스처 확장

```
import cv2
import numpy as np
import os
from PIL import ImageFont, ImageDraw, Image
from mediapipe.python.solutions import hands as mp_hands
from mediapipe.python.solutions import drawing_utils as mp_drawing
from mediapipe.python.solutions import drawing_styles as
mp_drawing_styles

def get_font(size=80):
    font_candidates = [
        "C:/Windows/Fonts/gulim.ttc",
        "C:/Windows/Fonts/malgun.ttf",
        "fonts/gulim.ttc",
    ]
    for path in font_candidates:
        if os.path.exists(path):
            return ImageFont.truetype(path, size)
    return ImageFont.load_default()

def detect_finger_states(hand_landmarks, handedness):
    """
    엄지: x좌표 비교 (오른손/왼손 구분)
    나머지: TIP.y < PIP.y 이면 펴진 것
    """
    lm = hand_landmarks.landmark
    fingers = []

    # 엄지 (오른손: TIP.x < IP.x, 왼손: TIP.x > IP.x)
    if handedness == "Right":
        fingers.append(1 if lm[mp_hands.HandLandmark.THUMB_TIP].x
            < lm[mp_hands.HandLandmark.THUMB_IP].x else 0)
    else:
        fingers.append(1 if
            lm[mp_hands.HandLandmark.THUMB_TIP].x >
            lm[mp_hands.HandLandmark.THUMB_IP].x else 0)

    # 검지, 중지, 약지, 새끼 (TIP.y < PIP.y → 펴짐)
    finger_tips = [
        mp_hands.HandLandmark.INDEX_FINGER_TIP,
        mp_hands.HandLandmark.MIDDLE_FINGER_TIP,
        mp_hands.HandLandmark.RING_FINGER_TIP,
        mp_hands.HandLandmark.PINKY_TIP,
    ]
    finger_pips = [
        mp_hands.HandLandmark.INDEX_FINGER_PIP,
        mp_hands.HandLandmark.MIDDLE_FINGER_PIP,
        mp_hands.HandLandmark.RING_FINGER_PIP,
        mp_hands.HandLandmark.PINKY_PIP,
    ]
}
```

양손 인식 및 제스처 확장

```
for tip, pip in zip(finger_tips, finger_pips):
    fingers.append(1 if lm[tip].y < lm[pip].y else 0)

return fingers # [엄지, 검지, 중지, 약지, 새끼]

def get_gesture_text(fingers):
    thumb, index, middle, ring, pinky = fingers
    count = sum(fingers)

    # 제스처 인식
    gestures = {
        (0, 0, 0, 0, 0): "주먹 (0)",
        (1, 0, 0, 0, 0): "엄지척 (1)",
        (0, 1, 0, 0, 0): "검지 (1)",
        (0, 1, 1, 0, 0): "가위 (2)",
        (1, 1, 0, 0, 0): "총 모양 (2)",
        (1, 1, 1, 1, 1): "보 (5)",
        (0, 1, 1, 1, 1): "넷 (4)",
        (0, 0, 0, 0, 1): "새끼손가락 (1)",
    }

    key = (thumb, index, middle, ring, pinky)
    return gestures.get(key, f"손가락 {count}개")

def draw_korean_text(image, text, x=50, y=50, font_size=60):
    if not text:
        return image

    font = get_font(font_size)
    image_pil = Image.fromarray(image)
    draw = ImageDraw.Draw(image_pil)

    try:
        bbox = draw.textbbox((x, y), text, font=font)
        w = bbox[2] - bbox[0]
        h = bbox[3] - bbox[1]
    except Exception:
        w, h = font.getsize(text)

    draw.rectangle((x - 5, y - 5, x + w + 15, y + h + 15),
        fill=(0, 0, 0, 180))
    draw.text((x + 5, y + 5), text, font=font, fill=(255, 255, 0))

    return np.array(image_pil)
```


양손 인식 및 제스처 확장

```
def main():
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("카메라를 열 수 없습니다.")
        return

    with mp_hands.Hands(
        max_num_hands=2,
        min_detection_confidence=0.7,
        min_tracking_confidence=0.5,
    ) as hands:

        while cap.isOpened():
            success, image = cap.read()
            if not success:
                continue

            image = cv2.cvtColor(cv2.flip(image, 1),
                                cv2.COLOR_BGR2RGB)
            image.flags.writeable = False
            results = hands.process(image)
            image.flags.writeable = True
            image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
            image_height, image_width, _ = image.shape

            if results.multi_hand_landmarks and
                results.multi_handedness:
                for hand_landmarks, hand_info in zip(
                    results.multi_hand_landmarks,
                    results.multi_handedness
                ):
                    # 오른손/왼손 판별 (미러링 반영)
                    handedness =
                        hand_info.classification[0].label

                    fingers =
                        detect_finger_states(hand_landmarks,
                                                handedness)
                    finger_count = sum(fingers)

                    gesture_text = get_gesture_text(fingers)
                    count_text = f"펴진 손가락:
                        {finger_count}개"
```


양손 인식 및 제스처 확장

```
# 손목 위치 기준으로 텍스트 위치 조정
wrist=hand_landmarks.landmark[mp_hands .HandLandmark.WRIST]
x_pos = int(wrist.x * image_width)
y_pos = max(int(wrist.y * image_height) - 120, 10)

hand_label = f"{'오른손' if handedness == 'Right' else '왼손'}"
image = draw_korean_text(image, hand_label, x=x_pos, y=y_pos, font_size=28)
image = draw_korean_text(image, gesture_text, x=x_pos, y=y_pos + 40, font_size=28)
image = draw_korean_text(image, count_text, x=x_pos, y=y_pos + 80, font_size=24)

mp_drawing.draw_landmarks(
    image,
    hand_landmarks,
    mp_hands.HAND_CONNECTIONS,
    mp_drawing_styles.get_default_hand_landmarks_style(),
    mp_drawing_styles.get_default_hand_connections_style(),
)

cv2.imshow("Hand Gesture Counter", image)
if cv2.waitKey(5) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

[예시 실행 결과]



양손 인식 및 제스처 확장

[코드 설명]

- `max_num_hands=2` : MediaPipe Hands가 한 프레임에서 최대 두 손까지 인식하도록 설정한 부분이다.
- `results.multi_hand_landmarks` : 검출된 여러 손의 랜드마크 정보를 저장하는 구조이다. 손이 두 개 인식되면 각 손의 랜드마크가 각각 독립적으로 저장된다.
- `results.multi_handedness` : 각 손이 왼손인지 오른손인지 판별하는 정보를 제공한다. 양손 인식에서는 이 정보를 이용해 손별 결과를 구분하여 출력할 수 있다.
- `for hand_landmarks, hand_info in zip(...)` : 랜드마크 정보와 손 방향 정보를 함께 묶어, 인식된 모든 손을 하나씩 순차적으로 처리하는 반복문이다. 즉, 양손을 각각 독립적으로 분석하는 핵심 구조이다.
- `handedness = hand_info.classification[0].label` : 현재 처리 중인 손이 "Left"인지 "Right"인지 판별하는 부분이다. 이를 이용해 화면에 왼손, 오른손을 구분하여 표시할 수 있다.
- `detect_finger_states(hand_landmarks, handedness)` : 각 손의 랜드마크 좌표를 이용하여 손가락의 펴짐 상태를 판단한다. 엄지는 왼손과 오른손에 따라 x좌표를 기준으로 판별하고, 검지·중지·약지·새끼는 TIP과 PIP의 y좌표를 비교하여 펴짐 여부를 판단한다.
- `finger_tips, finger_pips` : 각 손가락의 끝(TIP)과 중간 관절(PIP) 랜드마크를 리스트로 정리한 부분이다. 이를 통해 반복문으로 여러 손가락을 같은 방식으로 처리할 수 있어 코드가 간결해진다.
- `gestures = { ... }` : 손가락 상태와 제스처 이름을 연결하는 딕셔너리이다. (엄지, 검지, 중지, 약지, 새끼) 형태의 튜플을 키로 사용하여, 주먹·가위·보와 같은 손 모양을 문자열로 변환한다.
- `get_gesture_text(fingers)` : 현재 손가락 상태를 `gestures` 딕셔너리와 비교하여 해당 제스처를 반환한다. 정의되지 않은 경우에는 펴진 손가락 개수를 기반으로 결과를 출력한다.
- `finger_count = sum(fingers)` : 펴진 손가락의 개수를 계산하는 부분이다. 제스처와 함께 손가락 개수를 별도로 화면에 출력할 수 있게 한다.

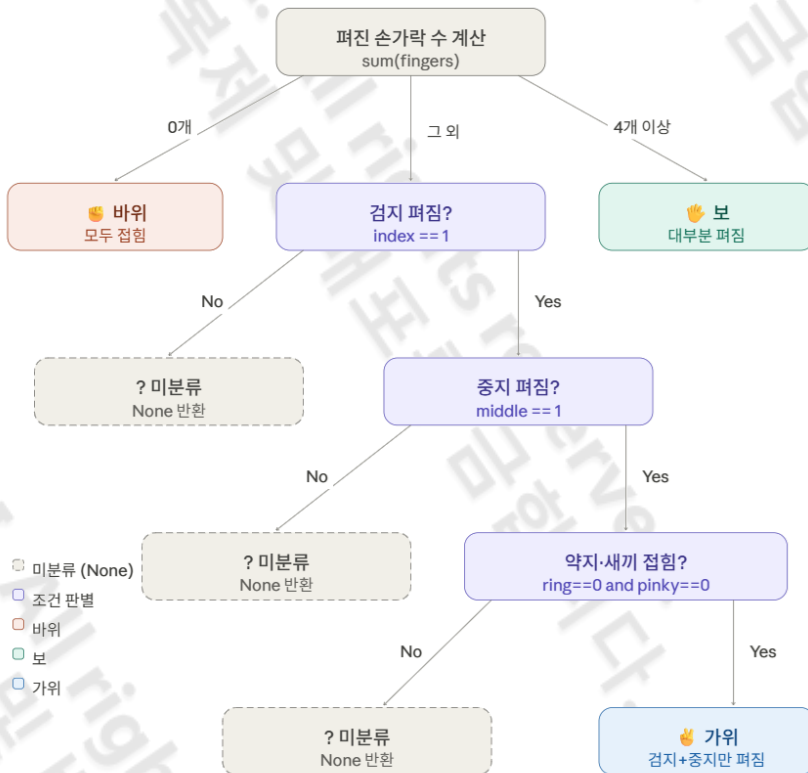
양손 인식 및 제스처 확장

- 손목 좌표 기반 출력 (WRIST) : 각 손의 손목 위치를 기준으로 텍스트 출력 위치를 정한다. 이를 통해 양손이 동시에 인식되더라도 결과가 각 손 근처에 구분되어 표시된다.
- `draw_korean_text(...)` : 왼손/오른손 표시, 제스처 이름, 손가락 개수를 한글 텍스트로 화면에 출력하는 함수이다. 실시간 인식 결과를 사용자가 쉽게 확인할 수 있도록 한다.
- `mp_drawing.draw_landmarks(...)` : 손의 랜드마크와 연결선을 화면에 시각적으로 표시하는 부분이다. 인식 결과를 눈으로 확인할 수 있게 한다.

이 코드는 웹캠 영상에서 손이 인식되면, 한 손만 처리하는 것이 아니라 검출된 모든 손을 반복문으로 하나씩 분석하는 방식으로 동작한다. 먼저 각 손의 랜드마크와 handedness 정보를 가져와 왼손과 오른손을 구분한 뒤, 손가락의 펴짐 상태를 판별하고, 그 결과를 바탕으로 손가락 개수와 제스처를 계산한다. 이후 계산된 결과는 각 손의 손목 위치를 기준으로 화면에 출력되며, 손 랜드마크와 연결선도 함께 표시된다. 따라서 사용자가 양손을 동시에 카메라 앞에 올리면, 왼손과 오른손이 각각 독립적으로 분석되고, 각 손의 제스처와 손가락 개수가 따로 화면에 나타나는 것을 확인할 수 있다. 즉, 이번 예제의 핵심은 한 손을 인식하는 단계에서 나아가, 두 손을 동시에 인식하고 각 손의 결과를 분리하여 처리할 수 있도록 확장했다는 점에 있다

• Rule-based AI 방식

제스처 분류 알고리즘은 손가락의 상태를 바탕으로 현재 손 모양이 어떤 의미를 가지는지를 판단하는 과정이다. 이때 본 예제에서는 사람이 직접 조건을 정의하고 그 조건에 따라 결과를 판별하는 Rule-based AI 방식을 사용한다. Rule-based AI는 입력된 데이터를 여러 단계의 조건으로 나누어 판단하는 구조를 가지며, 각 조건의 결과에 따라 다음 단계로 이동하는 방식으로 동작한다. 이러한 구조는 일반적으로 의사결정 트리(Decision Tree) 형태로 표현할 수 있다.



위 그림은 손가락 상태를 기반으로 가위, 바위, 보를 분류하는 과정을 단계적으로 나타낸 것이다. 이 과정은 다음과 같은 순서로 이루어진다. 먼저, 퍼진 손가락의 개수를 계산한다. 만약 손가락 개수가 0개라면 모든 손가락이 접힌 상태이므로 바위로 분류된다. 반대로 손가락 개수가 4개 이상이라면 대부분의 손가락이 펴진 상태로 판단하여 보로 분류한다.

이 두 조건에 해당하지 않는 경우에는 추가적인 조건 판단이 이루어진다. 먼저 검지

제스처 분류 알고리즘 설계

가 펼쳐 있는지를 확인하고, 검지가 펼쳐 있다면 다음 단계로 중지가 펼쳐 있는지를 확인한다. 이어서 약지와 새끼손가락이 접혀 있는지를 확인하여, 이 조건을 모두 만족하는 경우에 가위로 최종 분류한다. 이와 같은 조건을 만족하지 않는 경우에는 특정 제스처로 분류하지 않고 미분류(None) 상태로 처리한다.

• 제스처 조건 설계

이와 같은 분류 구조에서 중요한 점은 각 제스처를 명확한 조건으로 정의하는 것이다. 본 예제에서는 바위, 보, 가위를 다음과 같은 규칙으로 설계하였다. 바위는 모든 손가락이 접힌 상태로 정의되며, 손가락 개수가 0개인 경우로 판단한다. 보는 대부분의 손가락이 퍼진 상태로 정의되며, 손가락 개수가 4개 이상인 경우로 판단한다. 가위는 검지와 중지만 퍼지고 약지와 새끼손가락이 접힌 상태로 정의되며, 단계적인 조건 검사를 통해 판단한다.

이처럼 제스처는 단순한 패턴 인식이 아니라, 여러 조건이 순차적으로 결합된 논리적 규칙 구조로 설계된다.

• 오인식 개선

Rule-based AI는 조건이 명확하다는 장점이 있지만, 실제 환경에서는 손의 각도나 움직임에 따라 오인식이 발생할 수 있다. 예를 들어, 가위 제스처를 만들었다라도 약지가 약간 퍼지면 조건이 깨져 미분류로 처리될 수 있다.

이를 개선하기 위해서는 조건을 약간 완화하여 유사한 상태도 동일한 제스처로 인식하도록 설계할 수 있다. 예를 들어, 보의 경우 반드시 다섯 손가락이 모두 펼쳐야 하는 것이 아니라 네 개 이상 퍼진 경우도 포함하도록 설정할 수 있다. 또한 여러 프레임에서 동일한 결과가 반복될 때 최종 제스처로 확정하는 방식도 오인식을 줄이는데 효과적이다.

즉, Rule-based AI는 단순한 규칙 정의를 넘어, 실제 환경을 고려한 조건 설계가 함께 이루어져야 안정적인 성능을 얻을 수 있다.

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

- 실시간 양손 제스처 기반 가위·바위·보 대결 시스템

[실시간 양손 제스처 기반 가위·바위·보 대결 예시 코드]

```
import cv2
import numpy as np
import os
from PIL import ImageFont, ImageDraw, Image
from mediapipe.python.solutions import hands as mp_hands
from mediapipe.python.solutions import drawing_utils as mp_drawing
from mediapipe.python.solutions import drawing_styles as
mp_drawing_styles

def get_font(size=80):
    font_candidates = [
        "C:/Windows/Fonts/gulim.ttc",
        "C:/Windows/Fonts/malgun.ttf",
        "fonts/gulim.ttc",
    ]
    for path in font_candidates:
        if os.path.exists(path):
            return ImageFont.truetype(path, size)
    return ImageFont.load_default()

def detect_finger_states(hand_landmarks, handedness):
    """
    엄지: x좌표 비교 (오른손/왼손 구분)
    나머지: TIP.y < PIP.y 이면 펴진 것
    """
    lm = hand_landmarks.landmark
    fingers = []

    if handedness == "Right":
        fingers.append(1 if lm[mp_hands.HandLandmark.THUMB_TIP].x
                        < lm[mp_hands.HandLandmark.THUMB_IP].x else 0)
    else:
        fingers.append(1 if lm[mp_hands.HandLandmark.THUMB_TIP].x
                        > lm[mp_hands.HandLandmark.THUMB_IP].x else 0)

    finger_tips = [
        mp_hands.HandLandmark.INDEX_FINGER_TIP,
        mp_hands.HandLandmark.MIDDLE_FINGER_TIP,
        mp_hands.HandLandmark.RING_FINGER_TIP,
        mp_hands.HandLandmark.PINKY_TIP,
    ]
    finger_pips = [
```

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

```
mp_hands.HandLandmark.INDEX_FINGER_PIP,
mp_hands.HandLandmark.MIDDLE_FINGER_PIP,
mp_hands.HandLandmark.RING_FINGER_PIP,
mp_hands.HandLandmark.PINKY_PIP,
]

for tip, pip in zip(finger_tips, finger_pips):
    fingers.append(1 if lm[tip].y < lm[pip].y else 0)

return fingers # [엄지, 검지, 중지, 약지, 새끼]

def classify_rps(fingers):
    """
    손가락 상태를 가위/바위/보로 분류.
    반환: ('가위'|'바위'|'보'|None, 표시텍스트)
    """
    thumb, index, middle, ring, pinky = fingers
    count = sum(fingers)

    # 바위: 모두 접힘
    if count == 0:
        return "바위", "바위"

    # 보: 모두 펴짐 (엄지 포함 5개 or 엄지 없이 4개도 허용)
    if count >= 4:
        return "보", "보"

    # 가위: 검지 + 중지만 펴짐 (엄지 무관)
    if index == 1 and middle == 1 and ring == 0 and pinky == 0:
        return "가위", "가위"

    # 엄지척은 바위 계열로 처리
    if thumb == 1 and count == 1:
        return "바위", "바위"

    return None, f"? ({count}개)"

def judge_rps(shape1, shape2):
    """
    두 손의 가위바위보 판정.
    반환: ('왼쪽 승'|'오른쪽 승'|'무승부'|None)
    """
    if shape1 is None or shape2 is None:
        return None
```


실시간 양손 제스처 기반 가위·바위·보 대결 시스템

```
if shape1 == shape2:
    return "무승부"

# 이기는 관계: 가위>보, 바위>가위, 보>바위
wins = {
    "가위": "보",
    "바위": "가위",
    "보": "바위",
}

if wins[shape1] == shape2:
    return "왼쪽 승"
else:
    return "오른쪽 승"

def draw_korean_text(image, text, x=50, y=50, font_size=28, color=(255,
255, 0), bg_color=(0, 0, 0, 180)):
    if not text:
        return image

    font = get_font(font_size)
    image_pil = Image.fromarray(image)
    draw = ImageDraw.Draw(image_pil)

    try:
        bbox = draw.textbbox((x, y), text, font=font)
        w = bbox[2] - bbox[0]
        h = bbox[3] - bbox[1]
    except Exception:
        w, h = font.getsize(text)

    draw.rectangle((x - 5, y - 5, x + w + 15, y + h + 15),
fill=bg_color)
    draw.text((x + 5, y + 5), text, font=font, fill=color)

    return np.array(image_pil)

def draw_result_banner(image, result_text, color):
    """
    화면 상단 중앙에 결과 배너를 그림.
    """
    h, w, _ = image.shape
    font_size = 48
    font = get_font(font_size)
    image_pil = Image.fromarray(image)
    draw = ImageDraw.Draw(image_pil)
```


실시간 양손 제스처 기반 가위·바위·보 대결 시스템

```
try:
    bbox = draw.textbbox((0, 0), result_text, font=font)
    tw = bbox[2] - bbox[0]
    th = bbox[3] - bbox[1]
except Exception:
    tw, th = font.getsize(result_text)

x = (w - tw) // 2
y = 20
draw.rectangle((x - 10, y - 6, x + tw + 20, y + th + 16),
               fill=(0, 0, 0, 200))
draw.text((x + 5, y + 5), result_text, font=font,
         fill=color)
return np.array(image_pil)

def main():
    cap = cv2.VideoCapture(0)

    if not cap.isOpened():
        print("카메라를 열 수 없습니다.")
        return

    with mp_hands.Hands(
        max_num_hands=2,
        min_detection_confidence=0.7,
        min_tracking_confidence=0.5,
    ) as hands:

        while cap.isOpened():
            success, image = cap.read()
            if not success:
                continue

            image = cv2.cvtColor(cv2.flip(image, 1),
                               cv2.COLOR_BGR2RGB)
            image.flags.writeable = False
            results = hands.process(image)
            image.flags.writeable = True
            image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
            image_height, image_width, _ = image.shape

            hand_data = []

            if results.multi_hand_landmarks and
               results.multi_handedness:
                for hand_landmarks, hand_info in zip(
                    results.multi_hand_landmarks,
                    results.multi_handedness
                ):
                    :

```

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

```
handedness = hand_info.classification[0].label
fingers = detect_finger_states(hand_landmarks,
                                handedness)
shape, shape_text = classify_rps(fingers)

wrist = hand_landmarks.landmark[mp_hands.HandLandmark.WRIST]
x_pos = int(wrist.x * image_width)
y_pos = max(int(wrist.y * image_height) - 100, 10)

hand_data.append((x_pos, y_pos, shape, shape_text,
                  hand_landmarks, handedness))

# — 손 정보 표시 —
# 화면 x좌표 기준으로 왼쪽/오른쪽 정렬
hand_data.sort(key=lambda d: d[0])

labels = []
if len(hand_data) == 2:
    labels = ["1P", "2P"]
elif len(hand_data) == 1:
    labels = ["1P"]

for i, (x_pos, y_pos, shape, shape_text, hand_landmarks,
        handedness) in enumerate(hand_data):
    label = labels[i] if i < len(labels) else ""

    # 손 레이블 색: 1P=노랑, 2P=하늘
    text_color = (255, 255, 0) if i == 0 else (0, 255, 255)
    bg_color = (120, 0, 0, 180) if i == 0 else (0, 0, 120, 180)

    image = draw_korean_text(image, label, x=x_pos,
                              y=y_pos, font_size=26, color=text_color,
                              bg_color=bg_color)
    image = draw_korean_text(image, shape_text, x=x_pos,
                              y=y_pos + 38, font_size=28, color=text_color,
                              bg_color=bg_color)

    mp_drawing.draw_landmarks(
        image,
        hand_landmarks,
        mp_hands.HAND_CONNECTIONS,
        mp_drawing_styles.get_default_hand_landmarks_style(),
        mp_drawing_styles.get_default_hand_connections_style(),
    )
```

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

```
# — 가위바위보 판정 (손 2개일 때만) —
if len(hand_data) == 2:
    shape1 = hand_data[0][2]
    shape2 = hand_data[1][2]
    result = judge_rps(shape1, shape2)

    if result == "무승부":
        banner_text = "무승부!"
        banner_color = (200, 200, 200)
    elif result == "왼쪽 승":
        banner_text = "1P 승리!"
        banner_color = (80, 220, 255)
    elif result == "오른쪽 승":
        banner_text = "2P 승리!"
        banner_color = (80, 255, 160)
    else:
        banner_text = "인식 중..."
        banner_color = (180, 180, 180)

    image = draw_result_banner(image,
                               banner_text, banner_color)

elif len(hand_data) == 1:
    image = draw_result_banner(image, "손을 2개
                                보여주세요", (180, 180, 180))
else:
    image = draw_result_banner(image, "손을 2개
                                보여주세요", (180, 180, 180))

cv2.imshow("가위바위보 대결", image)
if cv2.waitKey(5) & 0xFF == 27:
    break

cap.release()
cv2.destroyAllWindows()

if __name__ == "__main__":
    main()
```

[예시 실행 결과]



실시간 양손 제스처 기반 가위·바위·보 대결 시스템

[코드 설명]

- `from PIL import ImageFont, ImageDraw, Image`
 - 한글 텍스트를 영상 위에 출력하기 위해 사용하는 라이브러리입니다.
 - OpenCV의 기본 `putText()` 함수는 한글 출력이 자연스럽게 않기 때문에, 이 코드에서는 PIL을 이용해 한글 레이블, 제스처 이름, 손가락 개수를 화면에 표시합니다.
 - 즉, 사용자가 결과를 더 직관적으로 볼 수 있게 만드는 역할을 합니다.
- `from mediapipe.python.solutions import hands as mp_hands`
 - MediaPipe Hands 기능을 사용하기 위한 부분입니다.
 - 이 모듈은 손의 위치를 찾는 것뿐 아니라, 손가락 마디와 끝점까지 포함한 21개의 랜드마크 좌표를 추출합니다.
 - 이후 손가락이 펴졌는지, 접혔는지, 어떤 제스처인지 판단하는 모든 과정은 이 랜드마크 좌표를 기반으로 이루어집니다.
- `def get_font(size=80):` : 한글 폰트를 불러오는 함수입니다
 - 여러 경로의 폰트 파일을 순서대로 확인한 뒤, 해당 폰트가 존재하면 그 폰트를 사용하고, 없으면 기본 폰트를 사용합니다.
 - 이 구조는 실행 환경에 따라 폰트 위치가 다를 수 있는 문제를 대비한 것입니다.
- `def detect_finger_states(hand_landmarks, handedness):` : 손가락이 펴졌는지 접혔는지를 판별하는 핵심 함수입니다.
- `lm = hand_landmarks.landmark` : 손의 21개 랜드마크를 더 간단하게 사용하기 위해 변수에 저장합니다.
- `fingers = [ ]` : 각 손가락 상태를 순서대로 저장할 리스트입니다.
 - 이 리스트에는 최종적으로 [엄지, 검지, 중지, 약지, 새끼] 순서로 0 또는 1이 저장됩니다.
- `if handedness == "Right": ~ else ~` : 엄지는 다른 손가락과 달리 좌우 방향으로 움직이는 경우가 많기 때문에 y좌표보다 x좌표 비교가 더 적절합니다.
 - 오른손이면 엄지 끝점이 엄지 마디보다 더 왼쪽에 있을 때 펴진 것으로 판단하고, 왼손이면 반대로 더 오른쪽에 있을 때 펴진 것으로 판단합니다.

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

- 검지, 중지, 약지, 새끼 판별 부분 : `finger_tips = [ ]`, `finger_pips = [ ]`, for tip, pip in `zip(finger_tips, finger_pips)`:
 - 나머지 네 손가락은 손가락 끝(TIP)과 중간 관절(PIP)의 y좌표를 비교합니다.
 - `TIP.y < PIP.y`이면 손가락 끝이 더 위에 있으므로 펴진 상태로 판단합니다.
- `return fingers` : 최종적으로 손가락 상태 리스트를 반환합니다.
 - 예를 들어 `[0, 1, 1, 0, 0]`이면 검지와 중지만 펴진 상태이고, `[1, 1, 1, 1, 1]`이면 모든 손가락이 펴진 상태입니다.
 - 이후 제스처 분류 함수는 바로 이 리스트를 바탕으로 동작합니다.
- `def classify_rps(fingers)` : 손가락 상태를 바탕으로 가위 / 바위 / 보를 판정하는 핵심 함수입니다.
- `thumb, index, middle, ring, pinky = fingers` : 각 손가락 상태를 따로 꺼내어 조건문에서 쉽게 사용합니다.
- `count = sum(fingers)` : 펴진 손가락 개수를 계산합니다. 손가락 개수와 조합을 함께 이용하여 손 모양을 분류합니다.
- `if count == 0` : 모든 손가락이 접힌 경우이므로 바위로 판별합니다.
- `if count >= 4` : 대부분의 손가락이 펴져 있으면 보로 판별합니다.
- `if index == 1 and middle == 1 and ring == 0 and pinky == 0` : 검지와 중지만 펴져 있으면 가위로 판별합니다.
- `if thumb == 1 and count == 1` : 엄지만 펴진 경우를 예외적으로 바위 계열로 처리하는 부분입니다.
- `return None, f"? ({count}개)"` : 규칙에 정확히 맞지 않는 손모양은 None으로 처리합니다
 - 이 처리는 매우 중요합니다.
 - 왜냐하면 애매한 손모양까지 억지로 가위·바위·보 중 하나로 분류하면 승패 판정이 틀릴 수 있기 때문입니다.
 - 따라서 불확실한 경우를 따로 처리함으로써 전체 시스템의 신뢰성을 높이고 있습니다.

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

- `def judge_rps(shape1, shape2)` : 두 손의 결과를 비교해 최종 승패를 정하는 함수입니다.
 - `shape1`과 `shape2`는 각각 첫 번째 손과 두 번째 손의 제스처 결과입니다.
 - 둘 중 하나라도 `None`이면 승패를 정하지 않습니다.
 - 두 손의 결과가 같으면 무승부입니다.
- `wins = {"가위": "보", "바위": "가위", "보": "바위"}` : 가위바위보의 승리 관계를 사전 형태로 정리한 부분입니다.
- `def draw_korean_text(...)` : 영상 화면 위에 한글 텍스트를 그리는 함수입니다.
 - 텍스트만 그리는 것이 아니라, 뒤에 반투명 배경 박스를 함께 그려서 글자가 더 잘 보이게 합니다.
 - 즉, 사용자가 카메라 화면 속에서도 손 이름, 제스처 결과, 플레이어 번호를 쉽게 읽을 수 있게 해 줍니다.
- `def draw_result_banner(image, result_text, color)` : 화면 상단 중앙에 최종 결과를 크게 표시하는 함수입니다.
 - 승리, 무승부, 인식 중 등의 결과를 배너 형식으로 띄워 게임 결과가 즉시 눈에 들어오도록 구성했습니다.
- `with mp_hands.Hands(...) as hands` : MediaPipe Hands 객체를 생성하는 부분입니다.
- `max_num_hands=2` : 최대 두 손까지 인식하도록 설정합니다.
- `min_detection_confidence=0.7` : 손을 처음 찾을 때 필요한 최소 신뢰도입니다.
- `min_tracking_confidence=0.5` : 이미 찾은 손을 계속 추적할 때 필요한 최소 신뢰도입니다.
 - 즉, 이 부분은 양손 인식 게임 환경에 맞는 정확도 기준을 설정한 것입니다.
- `image = cv2.cvtColor(cv2.flip(image, 1), cv2.COLOR_BGR2RGB)` : 프레임을 좌우 반전하고, BGR 영상을 RGB로 바꾸는 부분입니다.
 - 좌우 반전은 사용자가 화면을 거울처럼 보게 하여 손 움직임을 더 자연스럽게 느끼게 해 줍니다.

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

- RGB 변환은 MediaPipe가 RGB 입력을 사용하기 때문에 필요합니다.
- `image.flags.writeable = False` : MediaPipe에 영상을 넘기기 전에 프레임을 읽기 전용으로 바꾸는 부분입니다.
 - 이렇게 하면 메모리 복사와 수정이 줄어들어 처리 속도가 조금 더 좋아질 수 있습니다.
 - 실시간 영상 처리에서는 작은 최적화도 중요하기 때문에 의미 있는 코드입니다.
- `results = hands.process(image)` : 현재 프레임에서 손을 검출하고 랜드마크를 추출하는 핵심 처리입니다.
- `if results.multi_hand_landmarks and results.multi_handedness:` : 손 랜드마크와 손 방향 정보가 모두 있을 때만 다음 코드를 실행합니다.
- `for hand_landmarks, hand_info in zip(...)` : 인식된 여러 손에 대해 반복 처리하는 부분입니다.
 - 이 구조 덕분에 한 손만 인식하는 프로그램이 아니라 양손을 각각 독립적으로 분석하는 프로그램이 됩니다.
- `handedness = hand_info.classification[0].label` : 현재 손이 오른손인지 왼손인지 구분하는 부분입니다.
 - 이 값은 엄지 판별에서 매우 중요합니다.
 - 왜냐하면 엄지는 오른손과 왼손이 서로 반대 방향으로 보이기 때문에 같은 기준으로 판별하면 오인식이 발생할 수 있기 때문입니다.
- `fingers = detect_finger_states(hand_landmarks, handedness)` : 현재 손의 손가락 상태를 계산합니다.
- `shape, shape_text = classify_rps(fingers)` : 계산된 손가락 상태를 가위, 바위, 보 중 하나로 분류합니다.
- `wrist = hand_landmarks.landmark[mp_hands.HandLandmark.WRIST]` : 손목 좌표를 기준으로 사용하는 부분입니다.
 - 손목 좌표를 이용해 텍스트를 손 근처에 배치하면 각 손의 결과가 어느 손에 해당 하는지 더 쉽게 알 수 있습니다.

실시간 양손 제스처 기반 가위·바위·보 대결 시스템

- `hand_data.append(...)` : 각 손의 위치, 제스처 결과, 텍스트, 랜드마크, 손 방향 정보를 한 번에 저장합니다.
 - 이렇게 정보를 묶어 두면 나중에 정렬, 승패 비교, 화면 표시를 더 편하게 처리할 수 있습니다.
 - 즉, 이 리스트는 손 인식 결과를 관리하는 핵심 자료구조입니다.
- `hand_data.sort(key=lambda d: d[0])` : 손의 랜드마크와 연결선을 화면에 그리는 부분입니다.
 - 사용자는 카메라가 실제로 손을 어디까지 인식했는지 확인할 수 있고, 개발자는 오인식이 어느 손가락에서 발생했는지 분석할 수 있습니다.
- `if len(hand_data) == 2:` : 손이 두 개 인식되었을 때만 가위바위보 승패를 판정합니다.
 - 두 손의 제스처 결과를 꺼내 `judge_rps()`에 전달하고, 결과에 따라 무승부, 1P 승리, 2P 승리를 화면 상단에 배너로 표시합니다.
- `elif len(hand_data) == 1 / else` : 손이 하나만 있거나 손이 아예 없으면 “손을 2개 보여주세요” 같은 안내 문구를 출력합니다.

이 코드는 단순히 손의 위치를 인식하는 것을 넘어, 손가락의 상태를 분석하고 이를 기반으로 제스처를 판별한 뒤 최종 결과를 화면에 출력하는 하나의 통합된 처리 흐름으로 구성되어 있다. 먼저 MediaPipe를 통해 양손의 랜드마크를 추출한 후, 각 손의 손가락이 펴졌는지 접혔는지를 판단하여 이를 0과 1의 형태로 변환한다. 이 과정에서 엄지는 왼손과 오른손의 방향 차이를 고려하여 x좌표 기준으로 판별하고, 나머지 손가락은 y좌표를 기준으로 펴짐 여부를 판단함으로써 보다 정확한 인식이 가능하도록 설계되어 있다. 이렇게 계산된 손가락 상태는 가위, 바위, 보의 제스처로 분류되며, 단순한 개수 비교가 아니라 손가락의 조합을 함께 고려하여 보다 정교한 판단이 이루어진다. 이후 두 손이 동시에 인식된 경우에는 각 손의 제스처를 비교하여 승패를 결정하고, 그 결과를 직관적으로 확인할 수 있도록 화면 상단에 배너 형태로 출력한다.

또한 각 손의 위치를 기준으로 텍스트를 표시함으로써 사용자가 어떤 손이 어떤 결과를 의미하는지 쉽게 이해할 수 있도록 하였으며, 랜드마크를 시각적으로 함께 출력하여 인식 과정을 확인하고 오인식을 분석할 수 있도록 구성하였다

프로젝트 과제

- 손 제스처를 활용하여 다양한 기능을 제어하는 인터랙티브 프로그램을 만들어 보세요.

컴퓨터 비전과 MediaPipe를 활용하면 카메라 영상에서 손의 움직임을 인식하고, 이를 기반으로 다양한 프로그램을 제어할 수 있습니다. 지금까지 학습한 내용을 바탕으로, 손 제스처를 입력 장치로 활용하는 창의적인 프로그램을 설계해 보세요.

① 아이디어 구체화하기

다음과 같이 GPT에게 질문해보세요.

내가 만들 프로그램은 어떤 기능을 수행하는가?
손 제스처로 어떤 동작을 제어할 수 있는가?
이 프로그램은 어떤 문제를 해결하거나 어떤 재미를 줄 수 있는가?

+

🗣️

↑

- 가위바위보 게임 만들기
- 손가락 개수로 볼륨 조절하기
- 제스처로 영상 재생/정지 제어하기

② 제스처 설계하기

다음과 같이 GPT에게 질문해보세요.

어떤 손가락 조합을 어떤 동작으로 사용할까?
오인식을 줄이기 위해 조건을 어떻게 설정할까?
한 손과 두 손을 각각 어떻게 활용할 수 있을까?

+

🗣️

↑

- 바위 : 정지, 보 : 시작, 가위 : 모드 변경, 엄지 : 선택/확인 등과 같이 다양하게 변형

③ 프로그램 구조 설계하기

지금까지 작성한 코드 구조를 그대로 확장해보세요.

프로젝트 과제

- 카메라 입력 및 실시간 영상 처리
- 손 랜드마크 추출(MediaPipe)
- 손가락 상태 분석 (펴짐/접힘 판별)
- 제스처 분류 알고리즘 구현
- 제스처에 따른 기능 실행

④ UI/UX 구성하기

- 다음과 같이 GPT에게 질문해보세요.

제스처 인식 결과를 화면에 표시하기
현재 상태(모드, 결과 등)를 텍스트로 출력하기
사용자에게 안내 메시지 제공하기

+



⑤ 기능 확장하기 (심화)

- 다음과 같이 GPT에게 질문해보세요.

두 손을 이용한 게임 구현
제스처 정확도 개선 (임계값 조정)
다양한 제스처 추가 (OK, 엄지, 손바닥 등)
외부 기능과 연결 (음악, 웹, IoT 등)

+



© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.

© PMI. All rights reserved.
무단 복제 및 배포를 금합니다.